Chapter 7
Stochastic Optimization
Lectures on Monte Carlo Theory

Lorek & Rolski

Chapter Outline

Introduction: The Stochastic Optimization Problem I

In this chapter we survey Monte Carlo methods for a fundamental class of problems: given a function $h : \mathcal{S} \to \mathbb{R}$ defined on some set $\mathcal{S}$, find both the maximum value and the argument that achieves it.

Optimization Objective

$$\gamma^{\mathrm{opt}} = \max_{x \in \mathcal{S}} h(x), \qquad x^{\mathrm{opt}} = \arg\max_{x \in \mathcal{S}} h(x).$$

Here $\mathcal{S}$ is the **state space** or **solution space** (possibly combinatorial and enormous), h is the **performance function** whose maximum we seek, γ^{opt} (gamma superscript opt) is the optimal value, and x^{opt} is the optimizer.

When we want to *minimise* a cost function $H : \mathcal{S} \to \mathbb{R}_+$ (where $\mathbb{R}_+$ denotes the non-negative reals), we simply set $h = -H$. The minimiser of H is the same as the maximiser of $-H$, so all the maximisation algorithms apply directly. Many real-world problems—routing, scheduling, layout design, code-breaking—are naturally cast as cost minimisation.

Why use Monte Carlo and stochastic methods? For small $\mathcal{S}$ one can simply evaluate h at every point and return the maximum. But for combinatorial spaces—such as all permutations of n cities

Introduction: The Stochastic Optimization Problem II

(which has $n!$ elements) or all binary vectors of length d (which has 2^d elements)—exhaustive enumeration is completely infeasible. For example, the number of tours of $n = 20$ cities is $20! \approx 2.4 \times 10^{18}$; even at 10^9 evaluations per second this would take about 76 years. The key insight of stochastic optimisation is to *search intelligently* by constructing a random process that concentrates near the optimum.

The Monte Carlo bridge: Boltzmann distributions. The Boltzmann (or Gibbs) distribution—named after the physicist Ludwig Boltzmann—provides the bridge between probability and optimisation. For a cost function H and a temperature parameter $T > 0$ (tau, a positive real number representing how much randomness to allow):

$$\pi_v^T \;\propto\; e^{-H(v)/T}, \qquad v \in V.$$

The symbol $\propto$ (read "proportional to") means the right-hand side is divided by a normalising constant to make the probabilities sum to one. As $T \to 0$ (temperature approaches zero), the ratio $e^{-H(v)/T} / e^{-H(v^*)/T} = e^{-(H(v)-H(v^*))/T}$ tends to zero for any suboptimal v with $H(v) > H(v^*)$, so all probability mass concentrates on the minimiser(s) of H. This means: if we can efficiently sample from π^T for small T, we effectively locate the minimiser.

Introduction: The Stochastic Optimization Problem III

Five methods covered in this chapter.
The five stochastic optimisation methods we study form a natural progression from the simplest local-search approach to the most sophisticated distributional learning approach.

1. **Metropolis at fixed temperature.** Construct a Markov chain whose unique stationary distribution is the Boltzmann distribution π^T for a fixed $T > 0$. The mode of π^T is the minimiser of H, so the chain's most-visited state is approximately optimal. This is the simplest approach; it works well at moderate temperature but can be trapped in local minima.
2. **Simulated Annealing (SA).** Gradually decrease $T_k \searrow 0$ (T sub k decreases toward zero) while running the Metropolis chain. At high temperatures the chain explores freely and escapes local minima; as $T_k \to 0$ it settles near the global minimum. With a sufficiently slow *cooling schedule* $\{T_k\}$, convergence to the global optimum is guaranteed—this is the celebrated Hajék theorem.
3. **Locally Informed Proposals (LIP).** Improve each Metropolis step by evaluating the Boltzmann weights of *multiple* candidate neighbours before choosing one. The proposal is biased toward better solutions. The chain still targets π^T exactly (because a Metropolis–Hastings correction is applied), but moves toward better solutions far more often.

Introduction: The Stochastic Optimization Problem IV

4. **Cross-Entropy (CE) method.** Instead of maintaining a single current solution, maintain a *parametric distribution* $p_{\boldsymbol{\theta}}$ (p sub theta) over all of $\mathcal{S}$. Iteratively update the parameter $\boldsymbol{\theta}$ (theta) so that $p_{\boldsymbol{\theta}}$ concentrates on the best-performing solutions—the *elite set*. CE is borrowed from the rare-event estimation framework of Chapter 5.
5. **Metropolis–CE hybrid.** Combine the local Markov-chain walk of Metropolis with the distributional parameter learning of CE. At each step, generate M neighbours of the current solution and use the elite ones to update the *proposal distribution* so future steps propose better moves. This hybrid inherits the local focus of Metropolis and the adaptive quality of CE.

Introduction: The Stochastic Optimization Problem V

Two running benchmark problems.
Throughout the chapter all five methods are compared on two canonical combinatorial optimisation problems that together stress-test every algorithm.

Travelling Salesman Problem (TSP)

Given n cities with symmetric distance matrix $\mathbf{M}$ (entry $M(i,j) = M(j,i) =$ distance from city i to city j), find the permutation $\sigma^* \in \mathcal{S}_n$ (sigma, an ordering of cities, i.e., a complete tour visiting all cities exactly once) that minimises the total tour length:

$$l(\sigma) = \sum_{k=1}^{n-1} \mathbf{M}(\sigma(k), \sigma(k+1)) + \mathbf{M}(\sigma(n), \sigma(1)).$$

Here $\mathcal{S}_n$ is the set of all permutations of $\{1, \ldots, n\}$, which has $n!$ elements. For $n = 13$: $13! \approx 6.2 \times 10^9$ tours. The problem is NP-hard (no polynomial-time algorithm is known). The optimal tour length for our 13-city USA benchmark is **7024**, found by the Held–Karp dynamic-programming algorithm in 0.068 s. A uniformly random tour has expected length ≈ 15938—more than twice as long.

Introduction: The Stochastic Optimization Problem VII

0-1 Knapsack Problem

Given d items with weights $\mathbf{w} = (w_1, \ldots, w_d)$ and values $\mathbf{v} = (v_1, \ldots, v_d)$ (all non-negative) and a capacity W, choose a binary vector $\mathbf{x} \in \{0,1\}^d$ to

$$\text{maximise} \quad f(\mathbf{x}) = \mathbf{v} \cdot \mathbf{x} = \sum_{i=1}^{d} v_i x_i \quad \text{subject to} \quad \mathbf{w} \cdot \mathbf{x} = \sum_{i=1}^{d} w_i x_i \leq W.$$

Here $x_i = 1$ means item i is included; $x_i = 0$ means it is excluded. The state space is $\mathcal{S} = \{0,1\}^d$ with 2^d elements; for $d = 100$, $2^{100} \approx 10^{30}$. In our simulations: $d = 100$, $w_i = i$, $v_i = i^{1.2}$, $W = 3000$.

Key Insight

All five methods build a probability distribution that is higher on better solutions, then use Monte Carlo techniques to locate the distribution's mode. The methods differ in *how* they do the sampling efficiently.

7.1 Metropolis and Gibbs Based Methods: Setup I

We work with a graph $G = (V, E)$ where V is the set of candidate solutions and each edge connects two *neighbouring* solutions (solutions reachable in one local move). We are given a cost function $H : V \to \mathbb{R}_+$ and want to find $v^* = \arg\min_{v \in V} H(v)$.

The Boltzmann distribution. The core idea is to construct a probability distribution on V that assigns higher probability to states with lower cost. The *Boltzmann distribution* (named after physicist Ludwig Boltzmann, 1844–1906) at temperature $T > 0$ does exactly this:

Boltzmann Distribution (Eq. 7.1.1)

$$\pi_v^T = \frac{e^{-H(v)/T}}{\sum_{v' \in V} e^{-H(v')/T}}, \qquad v \in V, \quad T > 0.$$

Here $T > 0$ is the **temperature** parameter (higher T means more randomness and flatter distribution), $H(v) \geq 0$ is the **cost** of solution v, $e^{-H(v)/T}$ (e to the minus H of v divided by T) is the **Boltzmann weight** of v, and the denominator is the *partition function*—the normalising constant.

How to Read This Formula

When $H(v)$ is small (low cost, good solution), the exponent $-H(v)/T$ is close to zero, so $e^{-H(v)/T} \approx 1$—large weight. When $H(v)$ is large (high cost, bad solution), the exponent is very negative, so $e^{-H(v)/T} \approx 0$—small weight. Thus π_v^T is automatically larger for good solutions. As $T \to 0$, the difference between good and bad solutions is magnified: only the best solutions keep non-zero probability. At $T \to \infty$, all solutions are equally likely.

The key observation is that the Boltzmann distribution's mode is exactly the minimiser of H:

$$\arg\min_{v \in V} H(v) \;=\; \arg\max_{v \in V} \pi_v^T \quad \text{for any } T > 0.$$

Therefore, if we can sample from π^T and identify the most frequently visited state, we have found the optimum.

When we want to *maximise* a function $f : V \to \mathbb{R}_+$, we set $H(v) = -f(v)$, giving $\pi_v^T \propto e^{f(v)/T}$, which concentrates on the maximiser of f as $T \to 0$.

7.1 Metropolis and Gibbs Based Methods: Setup III

The Metropolis–Hastings algorithm for π^T. To sample from π^T via MCMC, we apply Metropolis–Hastings with the simple random walk on G as proposal: from state v, propose a uniformly random neighbour v' (so $q_{vv'} = 1/\deg(v)$ for $v' \in \mathcal{N}(v)$, where $\deg(v)$ is the number of neighbours of v). The Metropolis–Hastings acceptance probability is:

Acceptance Probability (Algorithm 61)

$$\alpha^T_{vv'} = \min\left(1,\ \frac{\pi^T_{v'} q_{v'v}}{\pi^T_v q_{vv'}}\right) = \min\left(1,\ \frac{\deg(v)}{\deg(v')} \cdot e^{(H(v)-H(v'))/T}\right), \quad v, v' \in V.$$

The resulting transition probability from v to neighbour v' is:

$$p^T_{vv'} = \frac{1}{\deg(v)} \min\left(1,\ \frac{\deg(v)}{\deg(v')} \cdot e^{(H(v)-H(v'))/T}\right), \qquad v' \in \mathcal{N}(v).$$

Here $\deg(v)$ (degree of v) is the number of neighbours of v in G. When the graph is *regular* (all degrees equal), the ratio $\deg(v)/\deg(v') = 1$ and the acceptance probability simplifies to $\min(1, e^{(H(v)-H(v'))/T})$.

7.1 Metropolis and Gibbs Based Methods: Setup IV

The interpretation is beautifully intuitive. A move to a *better* solution ($H(v') < H(v)$, i.e., lower cost) makes $H(v) - H(v') > 0$, so $e^{(H(v)-H(v'))/T} > 1$ and the move is *always* accepted (the min with 1 kicks in). A move to a *worse* solution ($H(v') > H(v)$) makes the exponent negative, so the acceptance probability is $e^{(H(v)-H(v'))/T} \in (0, 1)$—we accept the worse move with some probability. This occasional acceptance of worse moves is what prevents the chain from being permanently trapped in a local minimum.

Worked Example: Two-State Space

Suppose $V = \{A, B\}$ with $H(A) = 2$, $H(B) = 5$, $T = 1$, and the graph has edge A–B only. Starting from A:
To move to B (worse): $\alpha = e^{(2-5)/1} = e^{-3} \approx 0.05$. So 95% of the time the chain stays at A.
Starting from B, to move to A (better): $\alpha = \min(1, e^{(5-2)/1}) = \min(1, e^3) = 1$. The move is always accepted.
The stationary distribution is $\pi_A^T = e^{-2}/(e^{-2} + e^{-5}) \approx 0.95$, $\pi_B^T \approx 0.05$, confirming the chain spends far more time at the better solution A.

7.1 Metropolis and Gibbs Based Methods: Setup V

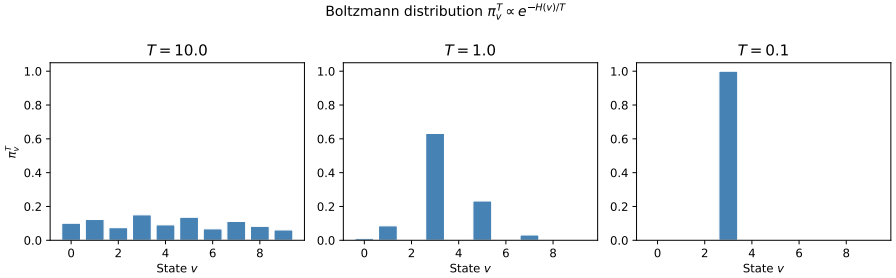


Figure: Boltzmann distribution for different temperatures. At high temperature T, all states receive nearly equal probability (flat distribution) and the chain explores freely. As T decreases, the distribution sharpens around the state with the smallest cost $H(v)$. In the limit $T \to 0$, the Boltzmann distribution becomes a point mass on the global minimiser. Fixed-temperature Metropolis samples from π^T for one chosen T, trading off exploration (high T) and exploitation (low T).

Application: Decrypting a Substitution Cipher I

A **substitution cipher** replaces each letter in the original message (plaintext) with a different letter according to a fixed permutation $\sigma : \Sigma \to \Sigma$, where $\Sigma = \{a, b, \ldots, z\}$ is the 26-letter alphabet. The ciphertext $c_1 c_2 \cdots c_L$ is produced by applying σ letter by letter: $c_i = \sigma^{-1}(\text{plaintext}_i)$. To decrypt, we need to recover σ.
The state space is $\mathcal{S} = \mathcal{S}_{26}$ (all permutations of 26 letters), with $26! \approx 4 \times 10^{26}$ elements—completely infeasible for exhaustive search.

The score function: bigram log-likelihood. English text has characteristic bigram (two-letter) frequencies: for example, "th" and "he" are very common, while "xz" is not. Let $\mathbf{M}(a, b)$ be the frequency with which letter b follows letter a in a reference English corpus (here: Tolstoy's *War and Peace*). For a candidate decryption key σ, the log-likelihood that the decrypted text is English is:

Application: Decrypting a Substitution Cipher II

Log-Likelihood Score (Eq. 7.1.2)

$$l(\sigma) = \sum_{i=1}^{L-1} \log \mathbf{M}(\sigma(c_i), \sigma(c_{i+1})).$$

Here c_i is the i-th ciphertext letter; $\sigma(c_i)$ is the candidate plaintext letter under key σ; $\mathbf{M}(\sigma(c_i), \sigma(c_{i+1}))$ is the reference bigram frequency; and log is the natural logarithm. A larger $l(\sigma)$ means the decrypted message looks more like English. We define the target distribution $\pi_\sigma \propto e^{l(\sigma)}$.

Why log-likelihood? Taking the logarithm converts a product of probabilities (one per bigram) into a sum, which is numerically stable and easier to compute. The maximiser of $l(\sigma)$ equals the maximiser of $\prod_i \mathbf{M}(\sigma(c_i), \sigma(c_{i+1}))$, which is the most likely key under the bigram model.

Proposal distribution. From permutation σ, propose σ' by *swapping* two uniformly chosen letters $a, b \in \{1, \ldots, 26\}$ (a transposition). There are $\binom{26}{2} = 325$ possible transpositions, each proposed with

Application: Decrypting a Substitution Cipher III

probability $q_{\sigma\sigma'} = 1/325$. Since the proposal is symmetric ($q_{\sigma\sigma'} = q_{\sigma'\sigma}$) and the graph is regular ($\deg(\sigma) = 325$ for all σ), the acceptance ratio simplifies to:

$$\alpha = \min\left(1,\ \frac{\pi_{\sigma'}}{\pi_\sigma}\right) = \min\left(1,\ e^{l(\sigma') - l(\sigma)}\right).$$

Crucially, this ratio can be computed *without* knowing the intractable partition function $z = \sum_\sigma e^{l(\sigma)}$. Only the *change* in log-likelihood due to the transposition matters, which can be computed in $O(L)$ time.

Application: Decrypting a Substitution Cipher IV

Simulation results for the substitution cipher. The algorithm was run for 3000 steps starting from a random permutation. The ciphertext was a paragraph from Tolstoy's *Anna Karenina*, encrypted with a randomly chosen permutation.

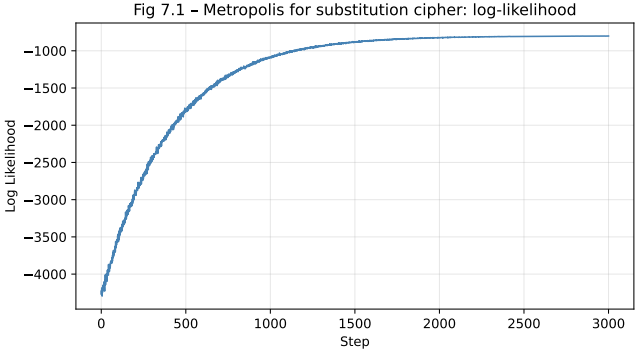


Table 7.1: Evolution of decrypted message.

Step	Log-likelihood
1	−4274.98
1000	−896.09
2000	−826.07
3000	−815.71

The log-likelihood improves by **3459** units in just 3000 steps. After step 3000 the message reads as recognisable English with only a few letters incorrect.

Application: Decrypting a Substitution Cipher V

The log-likelihood curve shows a rapid climb in the first few hundred steps (the chain finds the rough structure of the key quickly) followed by a slow refinement phase (fine-tuning individual letters). This two-phase behaviour is typical of MCMC-based optimisation.

Why it works. The Metropolis algorithm naturally concentrates its random walk on permutations that make the decrypted text look like English. The key advantage over brute force: the algorithm needs only the ratio $\pi_{\sigma'}/\pi_\sigma = e^{l(\sigma')-l(\sigma)}$, which is computed efficiently.

Key Insight

Metropolis on the substitution cipher works because the target distribution's ratio $\pi_{\sigma'}/\pi_\sigma$ can be computed cheaply without evaluating the intractable normalising constant z. This is a recurring theme in MCMC-based optimisation: we never need to know z, only the *ratio* of probabilities at two neighbouring states.

The 0-1 Knapsack Problem: Gibbs and Metropolis I

We want to find $\mathbf{x} \in \{0,1\}^d$ that maximises $f(\mathbf{x}) = \mathbf{v} \cdot \mathbf{x}$ subject to $\mathbf{w} \cdot \mathbf{x} \leq W$. We model this as sampling from a Boltzmann distribution that assigns zero probability to infeasible solutions.

Boltzmann Distribution for the Knapsack

$$\pi_{\mathbf{x}} = \begin{cases} \dfrac{\exp(\mathbf{v} \cdot \mathbf{x}/T)}{z_T} & \text{if } \mathbf{w} \cdot \mathbf{x} \leq W, \\ 0 & \text{if } \mathbf{w} \cdot \mathbf{x} > W. \end{cases}$$

Here $T > 0$ is the temperature, z_T is the normalising constant, and $\mathbf{v} \cdot \mathbf{x} = \sum_i v_i x_i$ is the total value of selected items. States that violate the weight constraint have zero probability and are never visited by the chain. As $T \to 0$, all mass concentrates on the highest-value feasible solution.

Gibbs sampler (Algorithm 63). In the Gibbs sampler we pick a random coordinate $i \in \{1, \ldots, d\}$ and resample x_i from its conditional distribution given all other coordinates $\mathbf{x}_{-i} = (x_1, \ldots, x_{i-1}, x_{i+1}, \ldots, x_d)$.

The 0-1 Knapsack Problem: Gibbs and Metropolis II

The ratio of Boltzmann weights for $x_i = 1$ vs $x_i = 0$ is $e^{v_i/T}$ (since only the i-th term $v_i x_i$ differs). Therefore the conditional probability of including item i takes the familiar *logistic* form:

$$\mathbb{P}(Z(i) = 1 \mid Z_{-i} = \mathbf{x}_{-i}) = \frac{e^{v_i/T}}{1 + e^{v_i/T}} = \frac{1}{1 + e^{-v_i/T}},$$

provided adding item i is feasible (i.e., $\mathbf{w} \cdot \mathbf{x}_{-i} + w_i \leq W$). If adding item i would exceed capacity, we are forced to set $x_i = 0$.

This conditional is a logistic (sigmoid) function of v_i/T: high-value items have large v_i/T making $e^{-v_i/T} \approx 0$, so $\mathbb{P}(x_i = 1) \approx 1$—the chain naturally includes them.

Worked Example: Gibbs Conditional

Suppose $v_i = 10$, $T = 1$. Then $\mathbb{P}(x_i = 1) = 1/(1 + e^{-10}) \approx 0.99995$. Item i is almost certainly included (if feasible).

With $v_i = 2$, $T = 1$: $\mathbb{P}(x_i = 1) = 1/(1 + e^{-2}) \approx 0.88$. With $v_i = 0.5$, $T = 1$: $\mathbb{P}(x_i = 1) = 1/(1 + e^{-0.5}) \approx 0.62$. Low-value items are included with near-50% probability.

The 0-1 Knapsack Problem: Gibbs and Metropolis III

Metropolis algorithm for Knapsack (Algorithm 64). Define neighbours of $\mathbf{x}$ as all binary vectors that differ in exactly one coordinate (bit-flips). Propose $\mathbf{x}'$ by flipping a uniformly random bit i. The ratio $\pi_{\mathbf{x}'}/\pi_{\mathbf{x}}$ is then:

$$\frac{\pi_{\mathbf{x}'}}{\pi_{\mathbf{x}}} = \exp\left(\frac{(1-2x_i)v_i}{T}\right) \cdot \mathbf{1}(\mathbf{w}\cdot\mathbf{x}' \le W),$$

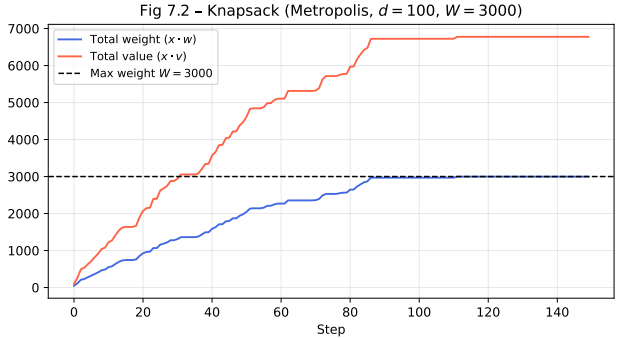
since only the i-th coordinate changes: $x'_j - x_j = (1-2x_i)\delta_{ij}$. This gives acceptance probability:

$$\alpha = \min\left(1,\, \exp\left(\tfrac{1-2x_i}{T} v_i\right)\right) \text{ if } \mathbf{w}\cdot\mathbf{x}' \le W, \quad \alpha = 0 \text{ otherwise.}$$

When $x_i = 0$ (adding item i): the exponent is $v_i/T \ge 0$, so $\alpha = 1$ (adding an item is always accepted if feasible). When $x_i = 1$ (removing item i): the acceptance probability is $e^{-v_i/T} < 1$—the chain resists removing valuable items.

Simulation results for $d = 100$, $w_i = i$, $v_i = i^{1.2}$, $W = 3000$, $T = 1$, running for 150 steps:

The 0-1 Knapsack Problem: Gibbs and Metropolis IV



The blue curve tracks total weight $X_k \cdot \mathbf{w}$ and the red curve tracks total value $X_k \cdot \mathbf{v}$. Both rise rapidly as the knapsack fills, then level off as the algorithm refines the selection.

After 150 steps: total weight ≈ 2977 (just below $W = 3000$), total value ≈ 6745.

Key Insight

Both Gibbs and Metropolis naturally respect the capacity constraint by assigning zero probability to infeasible states—no penalty term is needed. The logistic conditional in Gibbs makes high-value items preferentially included, giving the chain a built-in greedy tendency.

The Boltzmann Distribution: Intuition and Limiting Behaviour I

The Boltzmann distribution $\pi_v^T \propto e^{-H(v)/T}$ is the cornerstone of all algorithms in this chapter. Before proceeding to individual methods, it is worth understanding its behaviour at extreme temperatures through a concrete numerical example.

The Boltzmann Distribution: Intuition and Limiting Behaviour II

Worked Example: Three-State Space

Suppose $V = \{A, B, C\}$ with costs $H(A) = 1$, $H(B) = 3$, $H(C) = 7$. The Boltzmann weights at temperature T are $e^{-1/T}$, $e^{-3/T}$, $e^{-7/T}$ respectively, and the normalising constant is $z_T = e^{-1/T} + e^{-3/T} + e^{-7/T}$.

Temperature T	π_A^T	π_B^T	π_C^T
$T = 10$	0.461	0.334	0.205 (nearly uniform)
$T = 1$	0.659	0.239	0.101 (A preferred)
$T = 0.5$	0.841	0.113	0.046 (A strongly preferred)
$T = 0.1$	0.998	0.002	≈ 0 (point mass on A)

State A has the smallest cost $H(A) = 1$ and is the optimal solution. As $T \to 0$, $\pi_A^T \to 1$: the entire probability mass concentrates on the minimiser. At high T, the distribution is spread out and the chain can explore freely.

The Boltzmann Distribution: Intuition and Limiting Behaviour III

The Fundamental Trade-off

Every algorithm in this chapter must choose a temperature (or a schedule of temperatures). Too high: the chain explores everywhere but cannot identify the optimum. Too low: the optimum is identified but the chain gets stuck. The five methods differ in how they resolve this trade-off.

Algorithm 61: Metropolis for Boltzmann Distribution

```python
import numpy as np

def metropolis_boltzmann_step(v, neighbors, H, T, rng):
    """One Metropolis-Hastings step targeting Boltzmann pi^T (Alg. 61).
    v         : current state
    neighbors : dict mapping state -> list of neighbour states
    H         : cost function dict (H[v] = cost of state v)
    T         : temperature (float > 0)
    rng       : numpy RandomState
    """
    deg_v  = len(neighbors[v])
    v_prop = rng.choice(neighbors[v])          # propose a uniformly random neighbour
    deg_vp = len(neighbors[v_prop])
    # Acceptance: min(1, deg(v)/deg(v') * exp((H(v)-H(v'))/T))
    alpha = min(1.0, (deg_v / deg_vp) * np.exp((H[v] - H[v_prop]) / T))
    return v_prop if rng.uniform() <= alpha else v

def run_metropolis(v0, neighbors, H, T, n_steps, seed=0):
    """Run Metropolis-Boltzmann for n_steps; return trajectory."""
    rng   = np.random.RandomState(seed)
    v     = v0
    chain = [v]
    for _ in range(n_steps):
        v = metropolis_boltzmann_step(v, neighbors, H, T, rng)
        chain.append(v)
    return chain
```

Algorithms 63 & 64: Gibbs and Metropolis for Knapsack

```python
import numpy as np

def gibbs_knapsack_step(x, w, v, W, T, rng):
    """One Gibbs sampler step for 0-1 knapsack (Algorithm 63).
    x: current binary vector (length d), w,v: weight/value, W: capacity
    """
    d = len(x)
    i = rng.randint(0, d)                          # pick a random item
    weight_without_i = np.dot(w, x) - w[i] * x[i]
    if weight_without_i + w[i] > W:
        x[i] = 0                                   # forced to exclude: capacity exceeded
    else:
        # Conditional P(x_i=1) = 1/(1 + exp(-v_i/T)) -- logistic function
        prob1 = 1.0 / (1.0 + np.exp(-v[i] / T))
        x[i] = 1 if rng.rand() <= prob1 else 0
    return x

def metropolis_knapsack_step(x, w, v, W, T, rng):
    """One Metropolis step for 0-1 knapsack (Algorithm 64)."""
    d = len(x)
    i = rng.randint(0, d)
    x_new = x.copy()
    x_new[i] = 1 - x_new[i]                        # flip bit i
    if np.dot(w, x_new) <= W:                      # check feasibility
        # alpha=1 if adding item (x[i]=0), else exp(-v_i/T) if removing
        alpha = min(1.0, np.exp((1.0 / T) * (1 - 2 * x[i]) * v[i]))
        if rng.rand() < alpha:
```

7.2 Simulated Annealing: Motivation and Setup I

The dilemma of fixed temperature. In the Metropolis algorithm at a fixed temperature T, there is a fundamental trade-off between exploration and exploitation:

- At **high** T: the Boltzmann distribution π^T is nearly uniform, so the chain explores freely and escapes local minima easily. However, all states have nearly equal probability, so π^T puts little mass on the global minimum—we cannot easily identify the optimum from the chain's trajectory.
- At **low** T: π^T is sharply concentrated near the global minimum, which is exactly what we want. However, the chain mixes very slowly: it gets trapped in local minima because any move to a higher-cost state is accepted with probability $\approx e^{-\Delta H/T}$, which is tiny for small T and large energy barriers ΔH.

Neither extreme works. We need both exploration (to escape local minima) *and* exploitation (to converge to the global minimum).

The Simulated Annealing (SA) solution. SA resolves this dilemma by starting at a high temperature T_0 and gradually decreasing it according to a **cooling schedule** $T_0 \geq T_1 \geq T_2 \geq \cdots \searrow 0$. The name comes from metallurgy, where slowly cooling a heated metal (annealing) allows atoms to settle into a low-energy crystalline structure. Rapid cooling (quenching) produces a disordered, higher-energy glass-like structure; slow cooling produces a near-optimal crystal.

7.2 Simulated Annealing: Motivation and Setup II

At each step k, the chain uses the Metropolis transition probabilities corresponding to temperature T_k. This defines a *non-homogeneous* Markov chain: the transition matrix changes at every step.

Modified Boltzmann Distribution at Step k (Eq. 7.2.1)

$$\pi_v^{T_k} = \frac{\deg(v)\exp(-H(v)/T_k)}{z_{T_k}}, \qquad v \in V,$$

where $\deg(v)$ (degree of v) is the number of neighbours of v and $z_{T_k} = \sum_{v'} \deg(v')\exp(-H(v')/T_k)$ is the normalising constant at step k. The transition probability is:

$$p_{vv'}(k) = \frac{1}{\deg(v)} \min\left(1, \exp\left(-\frac{H(v') - H(v)}{T_k}\right)\right), \quad v' \in \mathcal{N}(v).$$

The acceptance probability for moving from v to neighbour v' at step k is $\alpha_k(v, v') = \min(1, e^{-(H(v')-H(v))/T_k})$: a better move ($H(v') < H(v)$) is always accepted; a worse move is accepted with probability $e^{(H(v)-H(v'))/T_k}$, which decreases as T_k decreases.

7.2 Simulated Annealing: Motivation and Setup III

Proposition 7.2.2: Convergence at fixed temperature. The following shows that taking $T \to 0$ concentrates mass on the global optimum. Let $D^* = \{v \in V : H(v) = \min_{v'} H(v')\}$ be the set of *optimal solutions*.

Proposition 7.2.2

For a non-homogeneous Markov chain with transition matrix (7.2.2):

$$\lim_{T \searrow 0} \lim_{\substack{k \to \infty \\ T_k = T}} \mathbb{P}(X_k \in D^*) = 1.$$

In words: for any fixed temperature T, the chain eventually visits optimal solutions with probability approaching 1 as $T \to 0$.

Proof sketch. Since the chain with fixed $T_k \equiv T$ is ergodic, its stationary distribution is π^{T_k}. Thus:

$$\lim_{\substack{k \to \infty \\ T_k = T}} \mathbb{P}(X_k \in D^*) = \sum_{v \in D^*} \pi_v^T = \frac{\sum_{v \in D^*} \deg(v) \exp(-H(v)/T)}{\sum_u \deg(u) \exp(-H(u)/T)}.$$

7.2 Simulated Annealing: Motivation and Setup IV

Dividing numerator and denominator by $\deg(v^*)\exp(-H(v^*)/T)$ and taking $T \to 0$: each term with $H(u) > H(v^*)$ vanishes since $e^{-(H(u)-H(v^*))/T} \to 0$, while terms with $H(u) = H(v^*)$ remain. Hence the limit equals 1. □

SA attempts to achieve this double limit by decreasing $T_k \to 0$ *simultaneously* with $k \to \infty$.

7.2 Simulated Annealing: Motivation and Setup V

How slow must the cooling be? Hajék's condition. Suppose v is a local minimum (not a global one). Let $c = \min_{v' \in \mathcal{N}(v)}(H(v') - H(v)) > 0$ be the *energy barrier*—the minimum cost increase needed to escape from v. The probability of the chain staying in v for one step satisfies:

$$p_{vv}(k) = 1 - \sum_{v' \in \mathcal{N}(v)} p_{vv'}(k) \geq 1 - e^{-c/T_k}.$$

If the chain visits v at step k, the probability it stays trapped in v forever is $\prod_{j=0}^{\infty} p_{vv}(k+j)$. The infinite product $\prod_{j=0}^{\infty}(1 - a_j) > 0$ if and only if $\sum_j a_j < \infty$ (a standard result from real analysis). Therefore:

Cooling Schedule Requirement (Hajék's Condition)

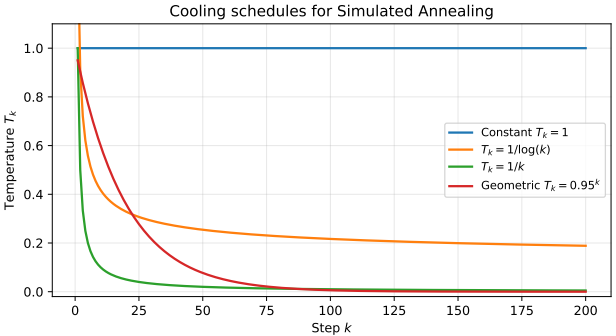
To guarantee the chain eventually escapes every local minimum, the cooling schedule must satisfy:

$$\sum_{k=0}^{\infty} e^{-c/T_k} = \infty.$$

If this sum converges, the chain can get permanently trapped in local minima. If it diverges, the chain will eventually escape.

The **logarithmic cooling schedule** $T_k = d/\log(k+2)$ satisfies Hajék's condition (since $e^{-c/T_k} = (k+2)^{-c/d}$ and $\sum_k (k+2)^{-c/d} = \infty$ for $c \leq d$) and provides a theoretical guarantee of convergence to the global optimum.
Geometric cooling $T_k = \beta^k T_0$ (with $0 < \beta < 1$, β pronounced "beta") gives $e^{-c/T_k} = \exp(-c/(\beta^k T_0))$ which decreases super-exponentially fast, so $\sum_k e^{-c/T_k} < \infty$ and *no* global convergence guarantee exists. However, geometric cooling is much faster in practice.

7.2 Simulated Annealing: Motivation and Setup VII



Common cooling schedules:

- **Constant** $T_k = T$: plain Metropolis, no cooling, no convergence guarantee
- **Logarithmic** $T_k = d/\log(k+2)$: theoretically valid; guarantees global optimum; very slow in practice
- **Geometric** $T_k = \beta^k T_0$: popular in practice; fast; no global guarantee
- **Linear** $T_k = T_0(1 - k/K)$: decreases too fast; usually fails

The fundamental requirement is $T_k \searrow 0$ slowly enough that $\sum_k e^{-c/T_k} = \infty$.

Checking Hajék's Condition for Logarithmic Cooling

With $T_k = 1/\log(k+2)$ (setting $d = 1$) and energy barrier $c = 1$:

$$e^{-c/T_k} = e^{-\log(k+2)} = \frac{1}{k+2}.$$

Since $\sum_{k=0}^{\infty} 1/(k+2) = \infty$ (harmonic series), the condition is satisfied and SA is guaranteed to find the global optimum. With geometric cooling $T_k = 0.99^k$: $e^{-1/T_k} = e^{-1/0.99^k} \to 0$ super-exponentially fast, so the sum converges and no guarantee.

Non-Homogeneous Markov Chains and SA I

Because the transition matrix of SA changes at every step, it is not a standard (time-homogeneous) Markov chain. We need the theory of **non-homogeneous Markov chains**.

Definition: Time Non-Homogeneous Markov Chain

A sequence (Y_n) is a **time non-homogeneous Markov chain** on a finite state space $\mathcal{S} = \{s_1, \dots, s_m\}$ if for any $s_{i_0}, \dots, s_{i_k} \in \mathcal{S}$:

$$\mathbb{P}(Y_0 = s_{i_0}, Y_1 = s_{i_1}, \dots, Y_k = s_{i_k}) = \mu_{s_{i_0}}\, p_{s_{i_0} s_{i_1}}(1)\, p_{s_{i_1} s_{i_2}}(2) \cdots p_{s_{i_{k-1}} s_{i_k}}(k),$$

where μ is the initial distribution and $p_{s_i s_j}(k) = \mathbb{P}(Y_k = s_j \mid Y_{k-1} = s_i)$ is the **time-dependent** transition probability at step k.

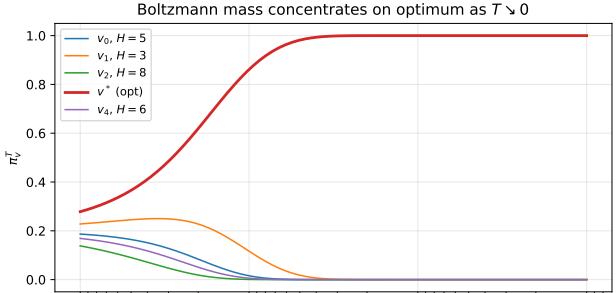
The distribution at time l is computed via matrix products:

Distribution at Time l (Eq. 7.2.3)

$$\big(\mathbb{P}(X_l = s_1), \ldots, \mathbb{P}(X_l = s_m)\big) = \boldsymbol{\mu}\,\mathbf{P}(0, l),$$

where $\mathbf{P}(k, l) = \mathbf{P}(k)\mathbf{P}(k+1)\cdots\mathbf{P}(l)$ is the product of transition matrices from step k to l. Note that $\mathbf{P}(k)$ here is the matrix whose (i, j) entry is $p_{s_i s_j}(k)$.

Remark 7.2.3. Simulated Annealing (Algorithm 65) is exactly a non-homogeneous Markov chain with transition probabilities $p_{vv'}(k) = q_{vv'}\alpha_k(v, v')$, where $q_{vv'}$ is the proposal probability and α_k is the Metropolis acceptance probability at temperature T_k.



As the temperature decreases along the cooling schedule, probability mass flows from high-cost states toward the optimal state(s). With logarithmic cooling, $\mathbb{P}(X_k \in D^*) \to 1$ as $k \to \infty$. The figure shows how the marginal distribution of the SA chain evolves over time as T_k decreases.

Algorithm 65: Simulated Annealing

```python
import numpy as np

def sa_step(v, neighbors, H, T_k, rng):
    """One SA step at temperature T_k (Algorithm 65).
    v         : current state
    neighbors : adjacency dict, neighbors[v] = list of neighbours
    H         : cost function dict
    T_k       : current temperature (decreases over time)
    """
    v_prop  = rng.choice(neighbors[v])
    delta_H = H[v_prop] - H[v]
    # Accept better moves always; worse moves with exp(-delta_H/T_k)
    alpha_k = min(1.0, np.exp(-delta_H / T_k))
    return v_prop if rng.uniform() <= alpha_k else v

def simulated_annealing(v0, neighbors, H, cooling_schedule, n_steps, seed=0):
    """Run SA with a given cooling schedule. Returns trajectory + best.
    cooling_schedule: list [T_0, T_1, ...] or callable k -> T_k
    """
    rng    = np.random.RandomState(seed)
    v      = v0
    traj   = [v]
    best_v = v
    for k in range(n_steps):
        T_k = (cooling_schedule[k]
               if hasattr(cooling_schedule, '__getitem__')
               else cooling_schedule(k + 1))
```

Remark 7.2.1: The Modified Boltzmann Distribution I

The Metropolis acceptance probability at temperature T_k uses the standard Boltzmann distribution $\pi_v^T \propto e^{-H(v)/T}$. However, when the graph G is *not* regular (i.e., different states have different numbers of neighbours), the transition matrix involves the degree ratio $\deg(v)/\deg(v')$. The **modified Boltzmann distribution** absorbs the degree into the stationary distribution:

Modified Boltzmann Distribution (Remark 7.2.1)

$$\tilde{\pi}_v^T = \frac{\deg(v)\, e^{-H(v)/T}}{\sum_{v' \in V} \deg(v')\, e^{-H(v')/T}}, \qquad v \in V.$$

Here $\deg(v)$ (degree of v, i.e., number of neighbours) appears as a pre-factor. This modified distribution is the *exact* stationary distribution of the Metropolis chain with transition probability $p_{vv'}^T = \frac{1}{\deg(v)} \min\!\left(1, \frac{\deg(v)}{\deg(v')} e^{(H(v)-H(v'))/T}\right)$ for $v' \in \mathcal{N}(v)$.

For *regular* graphs (all $\deg(v)$ equal), $\tilde{\pi}_v^T = \pi_v^T$: the degree factors cancel and the two distributions are identical. For non-regular graphs, the modified distribution places slightly more probability on

Remark 7.2.1: The Modified Boltzmann Distribution II

high-degree states. Since both distributions have the *same* mode (the minimiser of H), this does not affect the optimisation objective.

Practical consequence. When using Metropolis for optimisation on a *non-regular* graph, the standard implementation computes the acceptance ratio $\min(1, (\deg(v)/\deg(v')) \cdot e^{(H(v)-H(v'))/T})$, which automatically targets the modified Boltzmann distribution $\tilde{\pi}^T$. No special adjustment is needed; the degree ratio simply appears in the formula. For the substitution-cipher example, the graph is regular (every permutation has $\binom{26}{2} = 325$ neighbours), so the degree ratio equals 1 and the acceptance reduces to $\min(1, e^{l(\sigma')-l(\sigma)})$.

7.2 SA for the Travelling Salesman Problem I

In the TSP, the state space is $\mathcal{S} = \mathcal{S}_n$ (all tours of n cities). We minimise the tour length $l(\sigma)$, which we cast as a Boltzmann optimisation with $H(\sigma) = l(\sigma)$ and $\pi_\sigma^T \propto e^{-l(\sigma)/T}$.

Proposal distribution: transpositions. Two tours σ and σ' are *neighbours* if σ' is obtained from σ by swapping exactly two city positions i and j:

$$q_{\sigma\sigma'} = \frac{2}{n(n-1)} \quad \text{for each of the } \binom{n}{2} \text{ transpositions.}$$

Since the graph is regular (each tour has $\binom{n}{2}$ neighbours), the acceptance probability for moving from σ to σ' at step k is:

$$\alpha_k(\sigma, \sigma') = \min\left(1,\, e^{(l(\sigma)-l(\sigma'))/T_k}\right).$$

If the proposed tour σ' is shorter ($l(\sigma') < l(\sigma)$), the exponent is positive and $\alpha_k = 1$: the shorter tour is always accepted. If the proposed tour is longer, the exponent is negative and $\alpha_k = e^{(l(\sigma)-l(\sigma'))/T_k} \in (0, 1)$.

7.2 SA for the Travelling Salesman Problem II

SA for TSP: Transition Probability (Algorithm 67)

$$p_{\sigma\sigma'}(k) = \frac{2}{n(n-1)} \min\left(1,\, e^{(l(\sigma)-l(\sigma'))/T_k}\right), \quad \text{where } \sigma' \text{ differs from } \sigma \text{ by one transposition.}$$

2-opt variant (Remark 7.2.4). An often more effective proposal reverses a sub-segment of the tour. Choose indices $i < j$ uniformly at random and reverse the segment $\sigma(i), \sigma(i+1), \ldots, \sigma(j)$:

$$\sigma' = (\sigma(1), \ldots, \sigma(i-1), \underbrace{\sigma(j), \sigma(j-1), \ldots, \sigma(i)}_{\text{reversed}}, \sigma(j+1), \ldots, \sigma(n)).$$

This is the **2-opt move**. Two edges (u, v) and (u', v') that cross in the Euclidean plane are always replaced by non-crossing edges in the 2-opt step, typically reducing tour length. 2-opt often finds much shorter tours than transpositions because it can undo "crossed" edges directly.

7.2 SA for the Travelling Salesman Problem III

The 13-city USA benchmark: exact distance matrix.
The 13 USA cities benchmark uses the pairwise road distances (in miles) given in the 13×13 symmetric matrix **M** below, taken from the Google Operations Research routing benchmark. Entry $M(i, j) = M(j, i)$ is the road distance from city i to city j. The diagonal $M(i, i) = 0$ (a city has zero distance to itself).

We then have

$$\operatorname*{argmin}_{v\in V} H(v) = \operatorname*{argmax}_{v\in V} \pi_v^T.$$

If we are to find a maximum of some function $f : V \to \mathbb{R}_+$, then we may use above the Boltzmann distribution with cost function

$$H(v) = -f(v).$$

We will exchangeably use $\mathbf{x}$ or v for arguments of the functions we aim to maximize. In context of optimization they are called *states* or *solutions*. From the Metropolis-Hastings algorithm (see Proposition 6.3.2), to construct a Markov chain with stationary distribution π^T (we may sample from this distribution then) we must move from v to v' with probability $p_{vv'}^T = q_{vv'}\alpha_{vv'}^T$, where $\mathbf{Q} = (q_{vv'})$ is some candidate-generating matrix. Let us take $\mathbf{Q}$ corresponding to the simple random walk on the graph G as it was considered in Example 6.2.7. Assume that G is such that $\mathbf{Q}$ is regular. The acceptance probability is then given by

$$\alpha_{vv'}^T = \min\left(1, \frac{\deg(v)}{\deg(v')} e^{(H(v)-H(v'))/T}\right), \qquad v, v' \in V$$

and the transition probability is

$$p_{vv'}^T = \frac{1}{\deg(v)} \min\left(1, \frac{\deg(v)}{\deg(v')} e^{(H(v)-H(v'))/T}\right), \qquad v, v' \in V, q_{vv'} > 0.$$

Summarizing, the Metropolis-Hastings algorithm for Boltzmann distribution (with

SA for TSP: Algorithms 66 and 67 in Detail I

Recall the TSP setup: we have n cities and a symmetric distance matrix $\mathbf{M}$ with $M(i,j) = M(j,i) =$ distance from city i to city j. A tour is a permutation $\sigma \in \mathcal{S}_n$ and its length is $l(\sigma) = \sum_{k=1}^{n-1} M(\sigma(k), \sigma(k+1)) + M(\sigma(n), \sigma(1))$. We want to *minimise* $l(\sigma)$, equivalently to sample from $\pi_\sigma^T \propto e^{-l(\sigma)/T}$ (so $H(\sigma) = l(\sigma)$).

Algorithm 66: Fixed-temperature Metropolis for TSP.

Starting from a current tour $\sigma = Y_k$, one step of Metropolis proceeds as:

1. Choose two distinct indices $i, j \in \{1, \ldots, n\}$ uniformly at random (one of $\binom{n}{2}$ choices).
2. Define the proposed tour σ' by swapping $\sigma(i)$ and $\sigma(j)$: so $\sigma'(i) = \sigma(j)$, $\sigma'(j) = \sigma(i)$, and $\sigma'(k) = \sigma(k)$ for all other k.
3. Compute the acceptance probability: $\alpha = \min\big(1,\, e^{(l(\sigma) - l(\sigma'))/T}\big)$. Since the graph is regular ($\deg(\sigma) = \binom{n}{2}$ for all σ), the degree ratio is 1.
4. Generate $U \sim \mathcal{U}[0,1]$: if $U \leq \alpha$, set $Y_{k+1} = \sigma'$; otherwise $Y_{k+1} = Y_k$.

SA for TSP: Algorithms 66 and 67 in Detail II

Why the swap proposal? A transposition changes only two edges in the tour: the edges incident to positions i and j. If the swap reduces tour length ($l(\sigma') < l(\sigma)$), then $\alpha = 1$ and we always accept. If it increases tour length (the exponent is negative), we accept with probability $e^{(l(\sigma)-l(\sigma'))/T} \in (0,1)$, which decreases as T decreases.

Algorithm 67: Simulated Annealing for TSP.

Algorithm 67 is identical to Algorithm 66 except the temperature changes at every step: at step k (from time k to $k+1$), use temperature T_k from a cooling schedule $T_0 \geq T_1 \geq \cdots \searrow 0$. The acceptance probability at step k is:

$$\alpha_k = \min\left(1,\ e^{(l(\sigma)-l(\sigma'))/T_k}\right).$$

With the TSP cooling schedule $T_k = 1/\log(k+2)$ (logarithmic, satisfying Hajék's condition with $c \leq 1$ and $d = 1$), the SA chain is guaranteed to converge to the global minimum. With geometric cooling $T_k = 0.99^k \cdot T_0$, it converges faster in practice but without a global guarantee.

SA for TSP: Algorithms 66 and 67 in Detail III

Worked Example: One SA Step for TSP ($n = 4$)

Current tour: $\sigma = (1, 3, 2, 4)$ with $l(\sigma) = M(1,3) + M(3,2) + M(2,4) + M(4,1)$. Propose swap of positions 2 and 3: $\sigma' = (1, 2, 3, 4)$ with $l(\sigma') = M(1,2) + M(2,3) + M(3,4) + M(4,1)$. Compute $\Delta l = l(\sigma') - l(\sigma)$. If $\Delta l < 0$ (tour got shorter): accept always. If $\Delta l = +50$ (tour got longer by 50 miles) and $T_k = 100$: $\alpha = e^{-50/100} = e^{-0.5} \approx 0.607$. We accept the worse tour with 60.7% probability. At $T_k = 10$: $\alpha = e^{-5} \approx 0.007$—only 0.7% chance, showing how the chain increasingly resists worsening moves as temperature drops.

Simulation results for 13-city USA TSP. Running Algorithm 66 (constant $T = 1$) and Algorithm 67 ($T_k = 1/\log(k+2)$) for 100 steps each, starting from a uniformly random tour (average length ≈ 15938): SA finds better solutions in approximately **60%** of replications when both algorithms run for only 10 steps. Over 100 steps, both converge toward tour length $\approx$ 7024–9000, with SA tracking the lower values earlier in the run.

7.3 Locally Informed Proposals (LIP) I

The limitation of uninformed Metropolis. In the standard Metropolis algorithm, the proposal distribution $\mathbf{Q}$ is chosen independently of the target π: we propose a uniformly random neighbour. This is wasteful when many neighbours are clearly worse than the current state. For example, in the knapsack problem at a good solution, most bit-flips that remove valuable items will be rejected. We waste proposal evaluations on moves we will not accept.

The locally informed proposal (LIP) idea. We can do better by letting π *guide* the proposal. Rather than choosing among all neighbours uniformly, we evaluate the Boltzmann weight of each candidate neighbour and propose with probability proportional to $g(\pi_{s'}/\pi_s)$, where $g : (0, \infty) \to (0, \infty)$ is a *balancing function*. This concentrates proposals on directions where the target probability is higher. Let $\mathcal{N}(s) = \{s^{(1)}, \ldots, s^{(M)}\}$ be a set of M candidate neighbours of the current state s (sub-sampled from all neighbours). The **locally informed proposal distribution** is:

7.3 Locally Informed Proposals (LIP) II

LIP Distribution (Algorithm 68)

$$q_{ss^{(j)}}^{\mathrm{LIP}} = \frac{1}{Z_g}\, g\left(\frac{\pi_{s^{(j)}}}{\pi_s}\right) q_{ss^{(j)}}, \qquad j = 1, \ldots, M,$$

where $Z_g = \sum_{j=1}^{M} g\left(\frac{\pi_{s^{(j)}}}{\pi_s}\right) q_{ss^{(j)}}$ is the normalising constant ensuring the distribution sums to 1, $q_{ss^{(j)}}$ is the original (uninformed) proposal probability, and $g(\cdot)$ is the **balancing function**.

The most common choice is $g(r) = \sqrt{r}$ (square root), which satisfies the *balancedness condition* $g(r) = r \cdot g(1/r)$ (i.e., $\sqrt{r} = r \cdot \sqrt{1/r}$). This condition ensures the Metropolis–Hastings acceptance ratio takes a simple form.

7.3 Locally Informed Proposals (LIP) III

Metropolis–Hastings correction. Once a candidate X is proposed from q_s^{LIP}, the Metropolis–Hastings acceptance probability corrects for the bias:

$$\alpha = \min\left(1,\ \frac{\pi_X\, q_{Xs}^{\mathrm{LIP}}}{\pi_s\, q_{sX}^{\mathrm{LIP}}}\right).$$

When the balancing function satisfies $g(r) = r \cdot g(1/r)$, this simplifies to:

$$\alpha = \min\left(1,\ \frac{Z_g(s)}{Z_g(X)}\right),$$

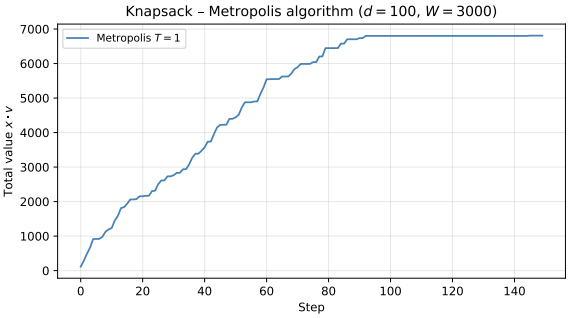
where $Z_g(s)$ and $Z_g(X)$ are the normalising constants at the current and proposed states respectively. The correction ensures π remains the exact stationary distribution: LIP is a valid MCMC algorithm.
Why $g(r) = \sqrt{r}$ works intuitively. If a neighbour s' has $\pi_{s'} \gg \pi_s$ (much better than current state), the weight $g(\pi_{s'}/\pi_s) = \sqrt{\pi_{s'}/\pi_s}$ is large, so s' is proposed more often. The square root is a compromise: it strongly prefers better neighbours but does not completely ignore worse ones, preventing degeneracy.

7.3 Locally Informed Proposals (LIP) IV

Partial LIP for computational efficiency. Computing $q_{ss'}^{\mathrm{LIP}}$ for *all* neighbours requires evaluating $g(\pi_{s'}/\pi_s)$ for every neighbour s'. For TSP, each tour has $\binom{n}{2} = 78$ neighbours for $n = 13$; for the knapsack, $d = 100$. Let $\mathfrak{n}$ denote the total number of neighbours. Set $M = \lceil \beta \mathfrak{n} \rceil$ for some fraction $\beta \in (0, 1]$ and randomly subsample M candidate neighbours. For $\beta = 1$: full LIP; for $\beta < 1$: faster approximation.

7.3 Locally Informed Proposals (LIP) V

Comparison on knapsack and TSP.



Knapsack ($d = 100$, $M = 100$ candidates):
LIP outperforms plain Metropolis because it preferentially proposes bit-flips that increase total value. The chain moves much more often in productive directions.

TSP (13 cities, $M = 100$):
In 100 replications, LIP won **9.66** times vs **0.33** for plain Metropolis. LIP finds shorter tours more reliably.

Cost: LIP requires evaluating π-ratios for M neighbours per step, vs 1 for plain Metropolis. The quality improvement usually justifies this.

7.3 Locally Informed Proposals (LIP) VI

Key Insight: LIP

LIP improves upon uninformed Metropolis by spending its "proposal budget" on the most promising directions. The balancing function $g(\sqrt{\cdot})$ ensures the algorithm is a valid Metropolis–Hastings step with the correct stationary distribution. LIP is a discrete analogue of the Metropolis-Adjusted Langevin Algorithm (MALA), which uses the gradient to guide proposals in continuous spaces.

Algorithm 68: LIP in Detail and Acceptance Ratio Derivation I

We now derive the Metropolis–Hastings acceptance ratio for LIP (Algorithm 68). Recall the LIP proposal at current state s: we generate M candidate neighbours $s^{(1)}, \ldots, s^{(M)}$ (either all neighbours or a fraction β of them) and propose state $s^{(j)}$ with probability:

$$q^{\mathrm{LIP}}_{s,s^{(j)}} = \frac{g(\pi_{s^{(j)}}/\pi_s)\, q_{s,s^{(j)}}}{Z_g(s)}, \qquad Z_g(s) = \sum_{l=1}^{M} g\left(\frac{\pi_{s^{(l)}}}{\pi_s}\right) q_{s,s^{(l)}}.$$

Suppose the proposal selects $X = s^{(j)}$ from the current state s. The Metropolis–Hastings acceptance probability is:

$$\alpha = \min\left(1,\ \frac{\pi_X \cdot q^{\mathrm{LIP}}_{X,s}}{\pi_s \cdot q^{\mathrm{LIP}}_{s,X}}\right).$$

Now compute $q^{\mathrm{LIP}}_{X,s}$ (the LIP probability of proposing s from X, over the candidate set evaluated at X):

$$q^{\mathrm{LIP}}_{X,s} = \frac{g(\pi_s/\pi_X)\, q_{X,s}}{Z_g(X)}.$$

Algorithm 68: LIP in Detail and Acceptance Ratio Derivation II

Substituting and using $q_{X,s} = q_{s,X}$ (symmetric proposal):

$$\frac{\pi_X \, q_{X,s}^{\mathrm{LIP}}}{\pi_s \, q_{s,X}^{\mathrm{LIP}}} = \frac{\pi_X \cdot g(\pi_s/\pi_X) \cdot q_{X,s}/Z_g(X)}{\pi_s \cdot g(\pi_X/\pi_s) \cdot q_{s,X}/Z_g(s)} = \frac{\pi_X \cdot g(\pi_s/\pi_X)}{\pi_s \cdot g(\pi_X/\pi_s)} \cdot \frac{Z_g(s)}{Z_g(X)}.$$

When $g(r) = \sqrt{r}$: $g(\pi_s/\pi_X) = \sqrt{\pi_s/\pi_X}$ and $g(\pi_X/\pi_s) = \sqrt{\pi_X/\pi_s}$, so:

$$\frac{\pi_X \cdot \sqrt{\pi_s/\pi_X}}{\pi_s \cdot \sqrt{\pi_X/\pi_s}} = \frac{\sqrt{\pi_X \pi_s}}{\sqrt{\pi_X \pi_s}} = 1.$$

Therefore the acceptance probability simplifies beautifully to:

Algorithm 68: LIP in Detail and Acceptance Ratio Derivation III

LIP Acceptance Ratio with $g(r) = \sqrt{r}$

$$\alpha = \min\left(1,\ \frac{Z_g(s)}{Z_g(X)}\right).$$

The acceptance probability depends only on the ratio of normalising constants at the current state s and the proposed state X. If the proposed state X has a larger local normalising constant $Z_g(X) > Z_g(s)$ (meaning its neighbourhood is richer/has higher π-values), the move is accepted with probability < 1; otherwise it is always accepted.

Intuition. The normalising constant $Z_g(s)$ is a measure of how "good" the neighbourhood of s looks. Moving to X is penalised if X has a better-looking neighbourhood, because we want to balance exploration and not get carried away to states that seem good but are surrounded by equally good neighbours.

Algorithm 68: LIP in Detail and Acceptance Ratio Derivation IV

Worked Example: LIP on 3-Neighbour Graph

Suppose $\pi_A = 0.5$, $\pi_B = 0.3$, $\pi_C = 0.1$, $\pi_D = 0.1$, and A's neighbours are $\{B, C, D\}$ (symmetric, uniform $q = 1/3$). At state A with $g(r) = \sqrt{r}$:

$$Z_g(A) = \sqrt{0.3/0.5}/3 + \sqrt{0.1/0.5}/3 + \sqrt{0.1/0.5}/3 = \tfrac{1}{3}(0.775 + 0.447 + 0.447) = 0.556.$$

The LIP proposal probabilities: $q^{\mathrm{LIP}}_{A,B} = 0.775/(3 \times 0.556) = 0.465$, $q^{\mathrm{LIP}}_{A,C} = q^{\mathrm{LIP}}_{A,D} = 0.268$. So B (the best neighbour) is proposed with 46.5% probability, vs 33.3% with uninformed Metropolis. LIP has successfully biased the proposal toward the better neighbour.

LIP for the TSP: Example 7.3.2 I

The LIP method can be applied to the TSP by evaluating the Boltzmann weight ratio $\pi_{\sigma'}/\pi_\sigma = e^{(l(\sigma)-l(\sigma'))/T}$ for each of the M candidate neighbours (transpositions) and then sampling proportionally to $\sqrt{e^{(l(\sigma)-l(\sigma'))/T}} = e^{(l(\sigma)-l(\sigma'))/(2T)}$.

Computational cost. In plain Metropolis for TSP, we evaluate the ratio for only 1 randomly chosen transposition. In LIP, we evaluate it for M transpositions (either all $\binom{n}{2}$ or a random subset of fraction β). For $n = 13$: $\binom{13}{2} = 78$ total transpositions. Full LIP ($\beta = 1$): evaluate all 78. Partial LIP ($\beta = 0.5$): evaluate ≈ 39.

Why this helps for TSP. In a random tour, many transpositions will shorten the tour (positive Δl) while others lengthen it. LIP preferentially proposes the length-reducing transpositions by sampling proportional to their Boltzmann weight. The MH correction with $\alpha = \min(1, Z_g(s)/Z_g(X))$ then ensures the target distribution remains exact.

LIP for the TSP: Example 7.3.2 II

LIP for TSP: Proposal Weight for Transposition (i, j)

Let $\sigma' = \tau_{ij}(\sigma)$ (tour with positions i and j swapped). The LIP weight for this transposition is:

$$w_{ij} = g\left(\frac{\pi_{\sigma'}}{\pi_\sigma}\right) = \sqrt{e^{(l(\sigma)-l(\sigma'))/T}} = e^{(l(\sigma)-l(\sigma'))/(2T)}.$$

The LIP proposal: choose transposition (i, j) with probability $w_{ij} / \sum_{i'<j'} w_{i'j'}$. For transpositions that shorten the tour ($l(\sigma') < l(\sigma)$): $l(\sigma) - l(\sigma') > 0$ so $w_{ij} > 1$, meaning they are overweighted. For lengthening transpositions: $w_{ij} < 1$, underweighted.

Simulation results from the book. Running 100 replications, 100 steps each, $T = 1$, $M = 100$ candidate neighbours:

- Plain Metropolis: found optimal 7024 in ≈ 0.33 of 100 runs.
- LIP ($M = 100$): found optimal 7024 in ≈ 9.66 of 100 runs.
- LIP is $\approx 29\times$ more likely to find the optimum per run.

LIP for the TSP: Example 7.3.2 III

The runtime trade-off: LIP evaluates $M = 100$ ratio computations per step vs 1 for Metropolis, so each step takes about $100\times$ longer. But for the same wall-clock time, LIP finds much better solutions because its proposal quality is dramatically higher.

Key Insight for LIP

LIP works because the Boltzmann weight ratio $\pi_{s'}/\pi_s$ is cheap to compute for each neighbour (it requires only the cost difference $H(s) - H(s')$), and the balancing function $g(\sqrt{\cdot})$ provides an exact MH correction. The quality gain from informed proposals far outweighs the cost of evaluating M ratios instead of 1.

7.4 The Cross-Entropy Method: Overview and Setup I

The Metropolis/SA/LIP methods are all **state-based**: they maintain a single current solution and make small local moves. The **Cross-Entropy (CE) method** takes a fundamentally different approach: it maintains a *distribution over the entire solution space* and iteratively shifts that distribution toward better solutions.

Setup. Let $h : \mathcal{S} \to \mathbb{R}$ be the performance function we want to maximise. Choose a parametric family of distributions $\{p_{\boldsymbol{\theta}} : \boldsymbol{\theta} \in \Theta\}$ on $\mathcal{S}$. Here $\boldsymbol{\theta}$ (theta) is the parameter vector describing the distribution. For example:

- **Knapsack:** independent Bernoulli coordinates, $p_{\boldsymbol{\theta}}(\mathbf{x}) = \prod_{j=1}^{d} \theta_j^{x_j}(1-\theta_j)^{1-x_j}$, $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_d) \in [0,1]^d$. Here $\theta_j \in [0,1]$ is the probability of including item j.
- **TSP:** a stochastic $n \times n$ matrix $\boldsymbol{\theta} = (\theta_{ij})$ with $\theta_{ii} = 0$ and $\sum_j \theta_{ij} = 1$. Here θ_{ij} is the probability of visiting city j immediately after city i in a sampled tour.

7.4 The Cross-Entropy Method: Overview and Setup II

We start with $\boldsymbol{\theta}_0$ (e.g., uniform: $\theta_j = 1/2$ for knapsack, $\theta_{ij} = 1/(n-1)$ for TSP) and iteratively update $\boldsymbol{\theta}$ so that $p_{\boldsymbol{\theta}}$ concentrates on the region where h is large.

Connection to rare-event estimation (Chapter 5). The CE method for optimisation is analogous to the CE method for rare-event estimation studied in Chapter 5. In **estimation**: $\boldsymbol{\theta}_0$ is given; the goal is to estimate $\mathbb{P}_{\boldsymbol{\theta}_0}(h(\mathbf{X}) \geq \gamma)$ for a fixed γ. In **optimisation**: $\boldsymbol{\theta}_0$ is our *choice*; we start from it and update toward γ^{opt}. No likelihood ratio weights are needed in optimisation ($W \equiv 1$) since we are not correcting for a change of measure.

7.4 The Cross-Entropy Method: Overview and Setup III

Algorithm 69: The CE method for optimisation.
The CE algorithm repeats the following two-step cycle until convergence:

1. **Update the elite threshold γ_t (gamma sub t).** Generate M samples $X_1, \ldots, X_M \sim p_{\theta'_{t-1}}$. Compute their performances $\eta_i = h(X_i)$ and sort them: $\eta_{(1)} \leq \eta_{(2)} \leq \cdots \leq \eta_{(M)}$. Set the elite threshold as the $(1-\rho)$-quantile:
$$\gamma_t = \eta_{(\lceil (1-\rho)M \rceil)}.$$
Here $\rho \in (0,1)$ (rho) is the *elite fraction* (e.g., $\rho = 0.1$ means the top 10% are elite). The notation $\lceil (1-\rho)M \rceil$ means round up to the nearest integer.
2. **Update the parameter θ'_t.** From the same M samples, identify the elite set $\mathcal{E}_t = \{X_i : h(X_i) \geq \gamma_t\}$ (all samples with performance at or above the elite threshold) and solve the maximum-likelihood problem:
$$\boldsymbol{\theta}' = \arg\max_{\boldsymbol{\theta}} \frac{1}{M} \sum_{i=1}^{M} \mathbf{1}(h(X_i) \geq \gamma_t) \log p_{\boldsymbol{\theta}}(X_i).$$

7.4 The Cross-Entropy Method: Overview and Setup IV

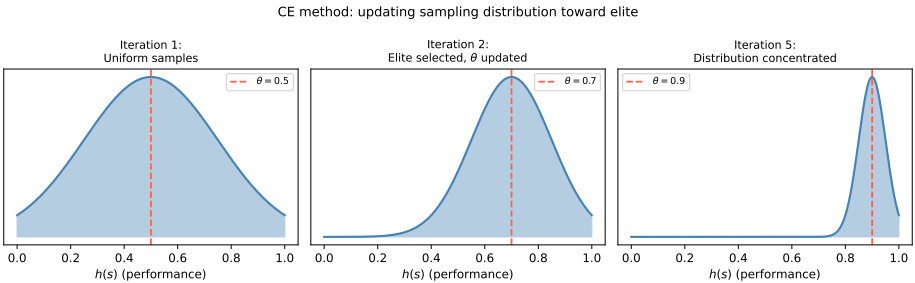
Apply a **smoothed update** to prevent premature convergence:

$$\boldsymbol{\theta}'_t = \alpha\, \boldsymbol{\theta}' + (1-\alpha)\, \boldsymbol{\theta}'_{t-1}.$$

Here $\alpha \in (0,1)$ (alpha) is the smoothing parameter (typically $\alpha = 0.7$): a larger α adapts faster; a smaller α is more stable and avoids collapsing to a degenerate distribution too quickly.

Stopping criterion: terminate when $\gamma_t = \gamma_{t-1} = \cdots = \gamma_{t-d}$ for some depth d (default $d = 5$), i.e., the elite threshold has not improved for d consecutive iterations.
Output: the best solutions found in the final elite set $\mathcal{E}$.

7.4 The Cross-Entropy Method: Overview and Setup V



At each iteration t, the distribution $p_{\boldsymbol{\theta}'_{t-1}}$ is sampled, the top ρ-fraction by performance forms the elite set (shaded), and the new parameter $\boldsymbol{\theta}'_t$ is fitted to the elites. The distribution shifts toward the high-performance region with each iteration.

CE for the Knapsack: Closed-Form Update Derivation I

For the knapsack, the parametric family is independent Bernoullis: $p_{\boldsymbol{\theta}}(\mathbf{x}) = \prod_{j=1}^{d} \theta_j^{x_j}(1-\theta_j)^{1-x_j}$. Taking the logarithm:

$$\log p_{\boldsymbol{\theta}}(\mathbf{x}) = \sum_{j=1}^{d} \big[x_j \log \theta_j + (1-x_j)\log(1-\theta_j)\big].$$

To find $\boldsymbol{\theta}'$, we differentiate the CE objective with respect to θ_k (theta sub k) and set to zero:

$$\begin{aligned} 0 &= \frac{\partial}{\partial \theta_k} \frac{1}{M} \sum_{i=1}^{M} \mathbf{1}(h(X_i) \geq \gamma_t) \log p_{\boldsymbol{\theta}}(X_i) \\ &= \frac{1}{M\theta_k(1-\theta_k)} \sum_{i=1}^{M} \mathbf{1}(h(X_i) \geq \gamma_t)(X_i(k) - \theta_k). \end{aligned}$$

Setting this to zero and solving for θ_k:

CE for the Knapsack: Closed-Form Update Derivation II

CE Update for Knapsack (Eq. 7.4.7)

$$\theta'_k = \frac{\sum_{i=1}^{M} \mathbf{1}(h(X_i) \geq \gamma_t)\, X_i(k)}{\sum_{i=1}^{M} \mathbf{1}(h(X_i) \geq \gamma_t)} = \text{fraction of elite samples that include item } k.$$

Here $X_i(k) \in \{0, 1\}$ is the k-th coordinate of sample X_i, $\mathbf{1}(h(X_i) \geq \gamma_t)$ is 1 if sample i is elite and 0 otherwise, and the formula computes the fraction of elite samples that have item k selected.

This is simply the **sample mean** of the elite samples' k-th coordinates. Item k gets a higher θ'_k if it appears frequently in the best solutions found so far. Over iterations: $\theta_k \to 1$ for items always in elite solutions; $\theta_k \to 0$ for items never in elite solutions.

CE for the Knapsack: Closed-Form Update Derivation III

Worked Example: 3-Item Knapsack CE Update

Suppose $d = 3$, $M = 5$ samples, elite threshold is top 40% (2 samples). The 2 elite solutions are: $\mathbf{x}^{(1)} = (1, 1, 0)$ (items 1 and 2 selected) and $\mathbf{x}^{(2)} = (1, 0, 1)$ (items 1 and 3 selected). Then:

$$\theta_1' = \frac{1+1}{2} = 1.0, \quad \theta_2' = \frac{1+0}{2} = 0.5, \quad \theta_3' = \frac{0+1}{2} = 0.5.$$

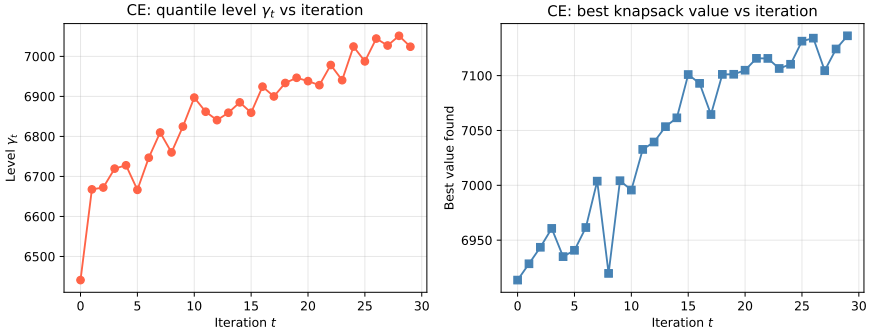
Item 1 appears in *both* elite solutions: $\theta_1' = 1$ (always include it). Items 2 and 3 each appear in exactly one: uncertain with 50% probability. After smoothing with $\alpha = 0.7$ and prior $\boldsymbol{\theta} = (0.5, 0.5, 0.5)$:

$$\boldsymbol{\theta}_t' = 0.7 \cdot (1, 0.5, 0.5) + 0.3 \cdot (0.5, 0.5, 0.5) = (0.85, 0.5, 0.5).$$

Without smoothing, θ_1' would jump straight to 1 and stay there forever—which is correct here but could be premature in general if only a few elite samples were available.

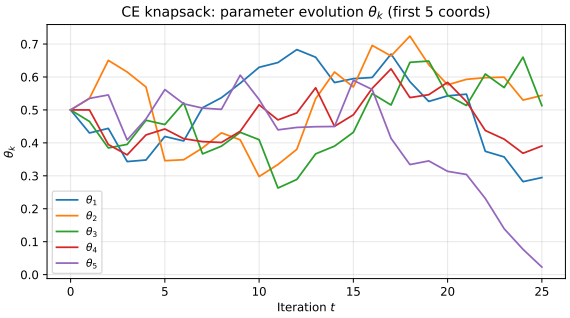
CE for the Knapsack: Closed-Form Update Derivation IV

Convergence of the CE method for the knapsack.



Left panel: the elite quantile γ_t increases monotonically over CE iterations, tracking the improvement in solution quality. **Right panel:** the best value found in each batch improves rapidly and then plateaus. CE typically converges in ≈ 30 iterations for $d = 100$. Each iteration uses $M = 200$ samples.

CE for the Knapsack: Closed-Form Update Derivation V



Evolution of θ_k parameters over CE iterations.

Each curve represents one coordinate θ_k (the probability of including item k). Over iterations:

- $\theta_k \to 1$: item k appears in every elite solution (the algorithm decides to always include it)
- $\theta_k \to 0$: item k never appears in elite solutions (the algorithm decides to never include it)

Items with the highest value-to-weight ratio v_i/w_i converge to $\theta_k = 1$ first.

Smoothing (with $\alpha = 0.7$) prevents premature convergence: without it, parameters would snap to 0 or 1 after just 1 iteration, permanently locking out potentially useful items.

CE for the Knapsack: Python Implementation

```python
import numpy as np

def ce_knapsack(d, w, v, W, theta0, M=200, rho=0.1, alpha=0.7,
                d_stop=5, max_iter=50):
    """Cross-Entropy method for 0-1 Knapsack (Algorithm 69).
    d: num items;  w,v: weight/value;  W: capacity
    theta0: initial Bernoulli param (shape d, e.g. all 0.5)
    M: sample size;  rho: elite fraction;  alpha: smoothing param
    """
    theta = theta0.copy()
    best_val, gamma_history = [], []
    for t in range(max_iter):
        # Step 1: sample M binary vectors from independent Bernoulli(theta)
        X = (np.random.rand(M, d) < theta).astype(float)
        # Compute performances (0 for infeasible solutions)
        h_vals = np.array([x @ v if x @ w <= W else 0.0 for x in X])
        # Step 2: compute (1-rho)-quantile = elite threshold gamma_t
        gamma_t = np.quantile(h_vals, 1 - rho)
        gamma_history.append(gamma_t)
        # Step 3: theta' = mean over elite samples (Eq. 7.4.7)
        elite_mask = h_vals >= gamma_t
        if elite_mask.sum() > 0:
            theta_new = X[elite_mask].mean(axis=0)  # fraction of elites with x_k=1
            theta = alpha * theta_new + (1 - alpha) * theta  # smoothed update
        best_val.append(h_vals.max())
        # Stop if elite level unchanged for d_stop consecutive iterations
        if (len(gamma_history) > d_stop and
```

CE for the TSP: Stochastic-Matrix Parametrisation I

For the TSP, we need a parametric family over permutations of n cities. The natural choice is a stochastic $n \times n$ matrix $\boldsymbol{\theta} = (\theta_{ij})$ with $\theta_{ii} = 0$ and $\sum_j \theta_{ij} = 1$ for all i.
Interpretation: θ_{ij} is the probability that city j is visited immediately after city i in a tour sampled from $p_{\boldsymbol{\theta}}$. Over iterations, as CE learns from elite tours, $\theta_{ij} \to 1$ for edges $(i \to j)$ that appear in the best tours and $\theta_{ij} \to 0$ for edges that never appear.

Algorithm 70: Sampling a Tour from $\boldsymbol{\theta}$

Fix the starting city as $\sigma_1 = 1$. For $k = 1, \ldots, n-1$:

1. Take row σ_k of $\boldsymbol{\theta}$: $\mathbf{r}^{k+1} = \boldsymbol{\theta}(\sigma_k, \cdot)$.
2. Zero out entries for already-visited cities: $r_j^{k+1} = 0$ for $j \in \{\sigma_1, \ldots, \sigma_k\}$.
3. Normalise: $q_j^{k+1} = r_j^{k+1} / \sum_i r_i^{k+1}$.
4. Sample $\sigma_{k+1} \sim \mathbf{q}^{k+1}$.

This greedy sequential sampling produces a valid tour (no repeated cities) because we zero out visited cities before sampling.

CE for the TSP: Stochastic-Matrix Parametrisation II

CE update for TSP (derivation). The log-probability of a tour $\sigma^{(i)} = (\sigma_1^{(i)}, \ldots, \sigma_n^{(i)})$ under $p_{\boldsymbol{\theta}}$ is (Eq. 7.4.8):

$$\log p_{\boldsymbol{\theta}}(X_i) = \sum_{k=1}^{n-1} \log \frac{\theta(\sigma_k^{(i)}, \sigma_{k+1}^{(i)})}{c(\sigma^{(i)}, k)},$$

where $c(\sigma^{(i)}, k) = \sum_{r \notin \{\sigma_1^{(i)}, \ldots, \sigma_k^{(i)}\}} \theta(\sigma_k^{(i)}, r)$ is the normalising constant at step k (sum of remaining row entries after zeroing visited cities).
Using Lagrange multipliers (maximising subject to row-sum constraints $\sum_j \theta_{ij} = 1$ for all i), the optimal θ'_{ab} is:

CE for the TSP: Stochastic-Matrix Parametrisation III

CE Update for TSP (Eq. 7.4.9)

$$\theta'_{ab} = \frac{\sum_{i=1}^{M} \mathbf{1}(h(X_i) \geq \gamma_t) \sum_{k=1}^{n-1} \mathbf{1}(\sigma_k^{(i)} = a,\, \sigma_{k+1}^{(i)} = b)}{\sum_{b'=1}^{n} \sum_{i=1}^{M} \mathbf{1}(h(X_i) \geq \gamma_t) \sum_{k=1}^{n-1} \mathbf{1}(\sigma_k^{(i)} = a,\, \sigma_{k+1}^{(i)} = b')}.$$

In words: θ'_{ab} is the fraction of times that the directed edge $a \to b$ appears in elite tours, among all edges leaving city a in elite tours. This row-normalised count ensures each row of $\boldsymbol{\theta}'$ sums to 1.

Intuition: if city j frequently follows city i in the best tours found so far, the next round of sampling will also tend to visit j after i. Over iterations, the matrix $\boldsymbol{\theta}$ converges to (approximately) the permutation matrix of the optimal tour.

Algorithm 71: CE for TSP – Full Procedure I

We now describe the complete CE algorithm for the TSP, combining sampling (Algorithm 70) with the parameter update (Eq. 7.4.9).

Setup. Let n be the number of cities, $\mathbf{M}$ the distance matrix, and $h(\sigma) = -l(\sigma)$ the performance function (negative tour length, so higher is better). The parameter $\boldsymbol{\theta}^{(0)}$ is initialised as a stochastic matrix with $\theta_{ij}^{(0)} = 1/(n-1)$ for $i \neq j$ and $\theta_{ii}^{(0)} = 0$: all transitions to unvisited cities are equally likely.

Algorithm 71: CE for TSP – Full Procedure II

Algorithm 71: CE for TSP

Require: $\boldsymbol{\theta}^{(0)}$, sample size M, elite fraction ρ, smoothing $\alpha \in (0, 1]$, max iterations T, stopping depth d.

For $t = 1, 2, \ldots, T$:

1. Sample M tours $X_1, \ldots, X_M$ using Algorithm 70 with $\boldsymbol{\theta}^{(t-1)}$.
2. Compute performances $\eta_i = h(X_i) = -l(X_i)$ for each tour.
3. Set $\gamma_t = \eta_{(\lceil (1-\rho)M \rceil)}$ (top-ρ quantile of $\{\eta_i\}$).
4. Compute raw update θ'_{ab} via Eq. 7.4.9 (edge-frequency count normalised by row).
5. Apply smoothing: $\boldsymbol{\theta}^{(t)} = \alpha \boldsymbol{\theta}' + (1 - \alpha)\boldsymbol{\theta}^{(t-1)}$.
6. **Stop** if γ_t unchanged for d consecutive iterations.

Return: best tour found in any sample across all iterations.

What the parameter $\boldsymbol{\theta}$ learns. Initially $\boldsymbol{\theta}^{(0)}$ is uniform (no preference). After the first iteration, edges that appear frequently in short tours (elite set) get higher weights. After many iterations, $\boldsymbol{\theta}^{(t)}$ converges

Algorithm 71: CE for TSP – Full Procedure III

to (approximately) the permutation matrix of the optimal tour: $\theta_{ij}^{(\infty)} = 1$ if city j follows city i in the optimal tour, and 0 otherwise. The smoothing parameter α prevents premature convergence: with $\alpha = 1$ (no smoothing) the matrix would snap to a single tour after the first iteration; with $\alpha = 0.7$ (default), it updates slowly and stably.

Convergence intuition for 4-city TSP

Suppose the optimal tour is $1 \to 3 \to 2 \to 4 \to 1$. Then over iterations: $\theta_{13} \to 1$, $\theta_{32} \to 1$, $\theta_{24} \to 1$, $\theta_{41} \to 1$, while all other entries $\to 0$. Sampling from this converged $\boldsymbol{\theta}$ almost certainly produces the optimal tour every time. The CE algorithm discovered the optimal tour structure by analysing which edges appear repeatedly in the best tours.

Simulation results (book, Sect. 7.6.1). With $M = 100$ tours per iteration, $\rho = 0.35$, $\alpha = 0.7$, running for 100 steps: CE found tour length ≈ 7300 (median over 100 replications), winning 61 of 100 replications. Runtime: $\approx 230\,\text{s}$ (versus $< 1\,\text{s}$ for plain Metropolis at 100 steps).

CE for TSP: Python Implementation

```python
import numpy as np

def sample_tour(theta, n, rng):
    """Algorithm 70: sample a tour from stochastic matrix theta."""
    sigma = [0]           # always start at city 0 (0-indexed)
    for k in range(n - 1):
        row = theta[sigma[-1]].copy()
        for already in sigma:
            row[already] = 0.0          # exclude already-visited cities
        row /= row.sum()                # normalise remaining probabilities
        next_city = rng.choice(n, p=row)
        sigma.append(next_city)
    return sigma

def tour_length(sigma, M_dist):
    """Total tour length for permutation sigma and distance matrix."""
    n = len(sigma)
    return (sum(M_dist[sigma[k], sigma[k+1]] for k in range(n-1))
            + M_dist[sigma[-1], sigma[0]])

def ce_tsp_update(elite_tours, n, alpha=0.7, theta_prev=None):
    """Compute theta' from elite tours via Eq. 7.4.9, with smoothing."""
    counts = np.zeros((n, n))
    for tour in elite_tours:
        for k in range(len(tour) - 1):
            counts[tour[k], tour[k + 1]] += 1  # count edge a->b occurrences
    row_sums = counts.sum(axis=1, keepdims=True)
```

7.5 The Metropolis Cross-Entropy: A Hybrid Approach I

Comparing the two paradigms.
Metropolis-based algorithms (Sections 7.1–7.3) maintain a *single current solution* $X^{(t)}$ and make small local moves guided by the target distribution. At each step, one new solution is evaluated.
The CE method (Section 7.4) maintains a *distribution parameter* $\boldsymbol{\theta}'_{t-1}$ over the *entire* solution space and generates M independent samples at each step. The state is a parameter, not a solution.

The Metropolis-CE hybrid idea (Algorithm 72). The Metropolis-CE algorithm combines both paradigms. The state at time t is simultaneously:

- A current solution $X^{(t)}$ (the Metropolis perspective), and
- A distribution parameter $\boldsymbol{\theta}'_{t-1}$ for the *neighbour proposal* distribution (the CE perspective).

At each step t, the algorithm:

1. Generates M *neighbours* of $X^{(t-1)}$ by sampling from the transition distribution $Q_{\boldsymbol{\theta}'_{t-1}}(X^{(t-1)}, \cdot)$.
2. Computes the elite threshold γ_t and identifies elite neighbours.
3. Updates $\boldsymbol{\theta}'_t$ using the CE maximum-likelihood update on the elite neighbours (maximising log-probability of elite neighbours under $Q_{\boldsymbol{\theta}}$).

7.5 The Metropolis Cross-Entropy: A Hybrid Approach II

4. Sets $X^{(t)}$ to the best-performing sample among all M neighbours.

Metropolis-CE Parameter Update (Eq. 7.5.1)

$$\boldsymbol{\theta}' = \arg\max_{\boldsymbol{\theta}} \frac{1}{M} \sum_{i=1}^{M} \mathbf{1}(h(X_i) \geq \gamma_t) \log Q_{\boldsymbol{\theta}}(X^{(t-1)}, X_i).$$

Key difference from CE: the likelihood $Q_{\boldsymbol{\theta}}(X^{(t-1)}, X_i)$ is the *transition probability from the current state* $X^{(t-1)}$ to X_i, not a global distribution over $\mathcal{S}$. This focuses learning on which *moves from the current position* are most productive.

7.5 The Metropolis Cross-Entropy: A Hybrid Approach III

Metropolis-CE for the Knapsack: derivation.
Define the transition distribution: from the current state $\mathbf{x} = X^{(t-1)}$, flip coordinate k with probability $\theta_k \geq 0$, where $\sum_k \theta_k = 1$ (i.e., $\boldsymbol{\theta}$ is a probability vector over the d possible bit-flips). A neighbour X_i produced by flipping coordinate a_i contributes $\log Q_{\boldsymbol{\theta}(X^{(t-1)}, X_i) = \log \theta_{a_i}}$.
The CE objective with constraint $\sum_k \theta_k = 1$ is maximised via the Lagrangian:

$$\mathcal{L}(\boldsymbol{\theta}, \lambda) = \frac{1}{M} \sum_{i=1}^{M} \mathbf{1}(h(X_i) \geq \gamma_t) \log \theta_{a_i} + \lambda \left(1 - \sum_{k=1}^{d} \theta_k \right).$$

Setting $\partial \mathcal{L} / \partial \theta_k = C_k / \theta_k - \lambda = 0$ gives $\theta_k = C_k / \lambda$. From $\sum_k \theta_k = 1$ we get $\lambda = \sum_j C_j$, so:

7.5 The Metropolis Cross-Entropy: A Hybrid Approach IV

CE Update for Knapsack Transitions

$$\theta'_k = \frac{C_k}{\sum_{j=1}^{d} C_j}, \qquad C_k = \sum_{i=1}^{M} \mathbf{1}(h(X_i) \geq \gamma_t) \cdot \mathbf{1}(\text{coordinate } k \text{ was flipped in } X_i).$$

Here C_k counts how often flipping bit k produced an elite neighbour. Intuitively: θ'_k is large if bit-flip k frequently leads to elite neighbours from the current state. The algorithm learns which bits are most profitable to flip *from where we are right now*.

The smoothed update is $\boldsymbol{\theta}'_t = \alpha\boldsymbol{\theta}' + (1-\alpha)\boldsymbol{\theta}'_{t-1}$. Running with $\rho = 0.3$, $M = 30$, $\alpha = 0.001$ for 100 steps: Metropolis-CE won **100 out of 100** replications on the knapsack.

7.5 The Metropolis Cross-Entropy: A Hybrid Approach V

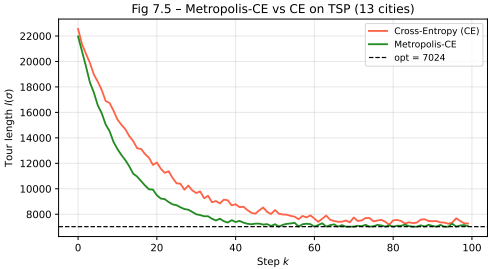
Metropolis-CE for the TSP.
For TSP, neighbours of the current tour $\sigma = X^{(t-1)}$ are obtained by swapping positions $i < j$ with probability $\theta(i,j)$. The parameter $\boldsymbol{\theta} = (\theta(i,j))_{i<j}$ is a probability vector over all $\binom{n}{2}$ possible swaps with $\sum_{i<j} \theta(i,j) = 1$.
The CE update is derived by the same Lagrangian argument. A neighbour X_l produced by swapping positions (a_l, b_l) contributes $\log Q_{\boldsymbol{\theta}(\sigma, X_l) = \log \theta(a_l, b_l)}$.

CE Update for TSP Swaps (Eq. 7.5.2)

$$\theta'(a,b) = \frac{C_{(a,b)}}{\sum_{i<j} C_{(i,j)}}, \quad C_{(a,b)} = \sum_{l=1}^{M} \mathbf{1}(h(X_l) \geq \gamma_t)\, \mathbf{1}(\text{swap } (a,b) \text{ was applied in } X_l).$$

Intuitively: $\theta'(a,b)$ becomes large if swapping positions a and b frequently produces elite solutions. Future proposals will preferentially try swaps that have historically been productive from the current tour.

7.5 The Metropolis Cross-Entropy: A Hybrid Approach VI



TSP results ($M = 100$, $\rho = 0.35$, $\alpha = 0.7$, 100 steps, 100 replications): CE won **61** times, Metropolis-CE won **29** times, LIP won **9.66** times, plain Metropolis won **0.33** times. Metropolis-CE achieves comparable quality to CE at roughly 23$\times$ lower computational cost.

Algorithm 72: Metropolis-CE Hybrid (Python)

```python
import numpy as np

def metropolis_ce_knapsack(x0, w, v, W, T=100, M=30, rho=0.3, alpha=0.001):
    """Metropolis-CE hybrid for 0-1 knapsack (Algorithm 72).
    x0: initial binary vector (length d)
    theta: probability vector over d bit-flips (which bit to flip)
    """
    d = len(x0)
    x_cur  = x0.copy()
    theta  = np.ones(d) / d          # initially uniform over all bit-flips
    best_x = x_cur.copy()
    best_h = x_cur @ v if x_cur @ w <= W else 0.0

    for t in range(1, T + 1):
        # Step 1: generate M neighbours by flipping bits according to theta
        flip_bits = np.random.choice(d, size=M, p=theta)
        neighbors = []
        for k_flip in flip_bits:
            nb = x_cur.copy()
            nb[k_flip] = 1 - nb[k_flip]   # flip bit k_flip
            neighbors.append((k_flip, nb))
        perfs = np.array([nb @ v if nb @ w <= W else 0.0 for (_, nb) in neighbors])
        # Step 2: compute elite threshold (top rho fraction)
        gamma_t    = np.quantile(perfs, 1 - rho)
        elite_mask = perfs >= gamma_t
        # Step 3: CE update -- count how often each bit-flip is elite
        if elite_mask.sum() > 0:
```

7.6 Summary of Simulation Results I

Experimental setup. All five algorithms were run on **100 replications** of each benchmark. For each replication, the algorithm ran for a fixed number of steps and we recorded the best solution found. A "win" means the algorithm found the strictly best solution among all five algorithms run with the same random seed. Win fractions sum to 100 (ties split equally).

7.6.1 Travelling Salesman Problem (13 cities)

The 13-city USA instance has optimal tour length **7024** (Held–Karp, 0.068 s). The TSP is NP-hard: for $n \geq 20$, brute force is infeasible. A uniformly random tour has average length ≈ 15938—more than twice the optimal.

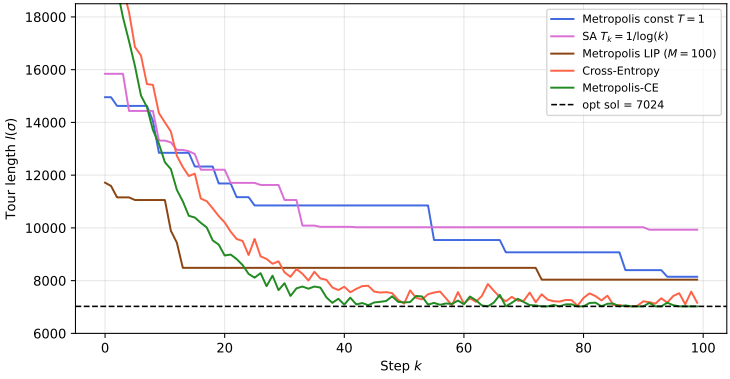
Algorithm	Win score	Approx. best (median)	Runtime (100 steps)
Metropolis ($T = 1$, constant)	0.33	~ 9000	< 1 s
Simulated Annealing ($T_k = 1/\log k$)	0.00	~ 8000	< 1 s
LIP ($M = 100$ candidates)	9.66	~ 7500	~ 10 s
Cross-Entropy (Alg. 71)	61.0	~ 7300	~ 230 s
Metropolis-CE (Alg. 72)	29.0	~ 7050	~ 10 s
Held–Karp (exact)	—	**7024**	0.068 s

7.6 Summary of Simulation Results II

Win scores are averages over 100 replications; they do not sum to exactly 100 because of fractional tie-splitting. The Held–Karp algorithm is exact but exponential in n; for $n = 13$ it runs in 0.068 s.

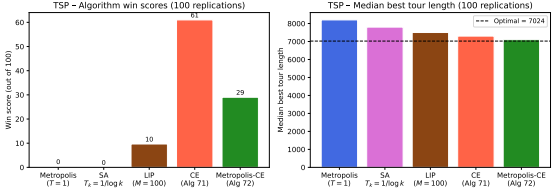
7.6 Summary of Simulation Results III

Fig 7.4 – 100 steps of various algorithms for TSP (13 cities)



7.6 Summary of Simulation Results IV

A single representative run of each algorithm over 100 steps (same random seed). The y-axis is the tour length; the dashed line at 7024 is the known optimum. Plain Metropolis (blue) gets trapped early. SA (magenta) makes rapid early progress by accepting bad moves. LIP (brown) converges faster due to informed proposals. CE (red) and Metropolis-CE (green) both converge very close to the optimum.



Win scores across 100 replications. CE-based methods dominate. CE wins most often (61/100) but requires $\approx 230\,\mathrm{s}$. Metropolis-CE wins 29 times with only $\sim 10\,\mathrm{s}$—a $23\times$ speed-up with comparable quality. LIP provides a useful middle ground (10 s, 9.66 wins).

7.6 Summary of Simulation Results V

7.6.2 The Knapsack Problem ($d = 100$)

For the knapsack ($d = 100$, $w_i = i$, $v_i = i^{1.2}$, $W = 3000$), all five algorithms were run 100 times for 100 steps each. The true optimal value is unknown analytically (NP-hard), but CE-based methods consistently achieve much higher values than uninformed methods.

Algorithm	Best value found (typical)	Win score
Metropolis ($T = 1$)	~ 6800	very low
Gibbs sampler ($T = 1$)	~ 6800	very low
LIP ($M = 100$)	~ 7100	moderate
Cross-Entropy	~ 7200	high
Metropolis-CE	~ 7400	**100/100**

For the knapsack, Metropolis-CE won **every single replication** (100/100). The reason: the adaptive transition distribution $\boldsymbol{\theta}$ learns which specific bit-flips improve the current solution from the current state. Each step is highly targeted. By contrast, uninformed Metropolis and Gibbs stall at ~ 6800 because they waste most steps on unprofitable moves—random bit-flips from a good solution mostly remove valuable items or add items that exceed the capacity.

7.6 Summary of Simulation Results VI

Key Takeaways from the Comparison

CE and Metropolis-CE outperform all Metropolis-based methods on both benchmarks. Metropolis-CE is $\approx 23\times$ faster than CE on TSP with comparable or better solution quality. LIP offers a useful middle ground. For practical applications: start with Metropolis to verify correctness, then try SA or LIP for better quality, and use CE or Metropolis-CE for the highest-quality solutions.

Understanding the Win Scores: How to Read the Comparison Table I

The "win score" reported for each algorithm is computed as follows. For each of the 100 replications (same problem, same start, different random seed), each algorithm is run and the best solution value is recorded. The algorithm with the strictly highest value (for knapsack) or strictly lowest tour length (for TSP) wins that replication outright (score: 1.0). If k algorithms tie for best, each receives $1/k$.

Why win scores don't sum to exactly 100. Ties (multiple algorithms finding the same best value) mean one replication contributes a fractional win to each tied algorithm. The win scores sum to 100 only if there are no ties.

The Knapsack result: why Metropolis-CE wins 100/100. The key reason Metropolis-CE dominates the knapsack is that it *learns* which specific bit-flips improve the current solution from the current position. Consider the situation after 50 steps: the knapsack is nearly full (weight ≈ 2900 out of 3000). The only profitable moves are to swap a low-value item currently included for a high-value item currently excluded. Uninformed Metropolis picks a random bit to flip: most flips either (a) remove a valuable item (rejected), (b) try to add an item that doesn't fit (infeasible, rejected), or (c) swap low for high (accepted). Only option (c) is productive, and it has low probability $\approx 1/d$. Metropolis-CE's adaptive $\boldsymbol{\theta}$ learns to concentrate probability on exactly these productive swaps, making nearly every step productive.

Understanding the Win Scores: How to Read the Comparison Table II

The TSP result: why CE wins most often. CE uses $M = 100$ fresh samples from the global distribution at every step, covering the whole tour space. Metropolis-CE only generates $M = 100$ neighbours of the current tour (local search). For TSP with $n = 13$, the local neighbourhood (all transpositions) has $\binom{13}{2} = 78$ elements. CE samples from a richer distribution and can "jump" to distant tours that no local method can reach in one step. This global coverage helps CE on the TSP.

Understanding the Win Scores: How to Read the Comparison Table III

Understanding the Speed-Quality Trade-off

The trade-off between the five methods can be summarised as:

Method	Quality (TSP wins/100)	Speed (steps/s)
Metropolis	0.33	$\sim 10^4$
SA	0.00	$\sim 10^4$
LIP	9.66	~ 100
CE	61.0	~ 1 (230 s for 100 steps)
Metropolis-CE	29.0	~ 10 (10 s for 100 steps)

Metropolis-CE achieves $29/61 \approx 48\%$ of CE's wins at $1/23$ of the cost: an excellent quality-per-second ratio.

7.7 Bibliographical Notes I

The field of stochastic optimisation draws on ideas from statistical mechanics, combinatorial optimisation, and information theory.

Metropolis algorithm. The Metropolis algorithm was introduced by Metropolis et al. (1953) in the context of statistical mechanics and generalised by Hastings (1970) to Metropolis–Hastings. The application to combinatorial optimisation via the Boltzmann distribution is classical. The application to **decryption of substitution ciphers** using bigram frequencies is described in Diaconis [24], which provides an accessible account of code-breaking via MCMC.

Simulated Annealing. SA was introduced independently by Kirkpatrick, Gelatt & Vecchi (1983) and Cerny (1985). The theoretical convergence result under logarithmic cooling—the condition $\sum_k e^{-c/T_k} = \infty$—is due to **Hajék [76]**, who proved that SA converges to the global optimum if and only if this condition holds. Algorithm 65 in this chapter is the one-step SA transition.

Locally Informed Proposals. LIP is inspired by Zanella (2020) on "informed proposals for discrete spaces" and by MALA (Metropolis-Adjusted Langevin Algorithm) for continuous distributions. The specific gradient-based approximation used here is from **Grathwohl et al. [71]**, titled *"Oops I Took a Gradient: Scalable Sampling for Discrete Distributions"*.

7.7 Bibliographical Notes II

Cross-Entropy method. The CE method for optimisation was developed by **Rubinstein [31]** and is comprehensively described in Rubinstein & Kroese, *The Cross-Entropy Method* (Springer, 2004). Algorithm 69 follows Algorithm 2.3 of that reference. The **FACE** algorithm (*Fully Automated Cross-Entropy*, Algorithm 4.1 in [31]) is a refined version where the sample size M adapts automatically.

TSP and the Held–Karp algorithm. The TSP is one of the most-studied NP-hard combinatorial problems. The Held–Karp dynamic-programming algorithm [82] (Held & Karp, 1962) runs in $O(n^2 2^n)$ time. The distance matrix (Fig. 7.3) was obtained from `https://developers.google.com/optimization/routing/tsp`.

Exercise 7.L.1 (Substitution Cipher Decryption)

Implement the Metropolis-based decryption algorithm for a substitution cipher. Helper files: `ch7_cipher_decrypt_metropolis.py`, `ch7_cipher_encrypt.py`, `ch7_cipher_initial_permutation.py`, and Tolstoy's *War and Peace* for building the bigram matrix $\mathbf{M}$.

Run for 3000 steps from a random permutation. Report:

(a) The evolution of log-likelihood $l(\sigma_k)$ over steps.

(b) The decrypted messages at steps 1000, 2000, 3000.

Exercise 7.L.2 (Metropolis for Permutation Distributions)

Consider the probability distribution on $\mathcal{S}_n$:

$$\pi_\sigma = \frac{1}{z}\lambda^{d(\sigma,\sigma_{\mathrm{id}})}, \quad \lambda > 0, \quad d(\sigma,\sigma') = \frac{1}{n}\sum_{i=1}^{n} |\sigma(i) - \sigma'(i)|,$$

where $\sigma_{\mathrm{id}} = (1, 2, \ldots, n)$ is the identity permutation and $d(\sigma, \sigma')$ is the normalised ℓ_1-distance between two permutations.

(a) Implement the Metropolis algorithm using the TSP proposal (Algorithm 66) for $\lambda = 0.75$, $n = 52$. For which values of λ is the algorithm applicable?

(b) Repeat using the Kendall τ distance instead of $d(\sigma, \sigma')$.

(c) Use the 2-opt proposal (Remark 7.2.4) and compare convergence.

7.8 Selected Exercises III

Exercise 7.L.3 (Fixed-Point Distribution on Permutations)

Consider $\pi_\sigma = \lambda^{\mathrm{FP}(\sigma)}/z$, where $\mathrm{FP}(\sigma) = \#\{i : \sigma(i) = i\}$ is the number of fixed points and $\lambda > 0$. For $n = 52$ and $\lambda \in \{0.1, 0.75, 2\}$: implement the Metropolis algorithm and compare with the acceptance-rejection method from Exercise 3.L.6.

Exercise 7.L.4 (Knapsack: Varying Capacity)

Re-implement the Gibbs sampler for the knapsack (Algorithm 63) with $d = 100$, $w_i = i$, $v_i = i^{1.2}$ and capacities $W \in \{50, 500, 1000, 3000\}$.

(a) For each W, estimate the proportion of random binary vectors that are feasible. Compare the best values found by Gibbs vs random sampling.

(b) Implement Metropolis for the same problem and compare.

Exercise 7.L.5 (Simulated Annealing for Knapsack)

Construct an SA version of the knapsack Gibbs sampler:

$$\pi_{\mathbf{x}}^{T_k} = \frac{\exp(\mathbf{v}\cdot\mathbf{x}/T_k)}{z_{T_k}}, \quad \mathbf{w}\cdot\mathbf{x} \le W.$$

Compare for $W \in \{3000, 50\}$: fixed temperature $T_k = 1$ vs logarithmic cooling $T_k = 1/\log(k)$. Note: by Remark 7.2.1, adding $\deg(\mathbf{x})$ to the Boltzmann weight gives the modified distribution that is stationary for the Gibbs chain.

Exercise 7.L.6 (CE for TSP: Alternative Parametrisation)

In Sect. 7.4 the TSP used a stochastic matrix $\boldsymbol{\theta} = (\theta_{ij})$ $(n \times n)$. Consider instead a *vector* parametrisation $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_n)$ (a probability distribution over cities).

(a) Describe formally how to sample a tour from this vector: at each step, given that cities $c_1, \ldots, c_k$ have been visited, zero out their probabilities, renormalise, and sample the next city.

(b) Implement the full CE method using both parametrisations for $n = 13$ and the distance matrix $\mathbf{M}$ from Fig. 7.3.

(c) Compare convergence speed and solution quality. Does the matrix parametrisation capture more structure of the TSP than the vector parametrisation?

Chapter 7 Summary I

This chapter covered five Monte Carlo methods for stochastic optimisation of combinatorial problems. Each method builds on the Boltzmann distribution as the bridge between probability and optimisation, but differs in how sampling is performed.

Five Stochastic Optimization Methods: Side-by-Side Comparison

Method	State	Core mechanism	Key parameters
Metropolis (fixed T)	single solution	Boltzmann stationary dist. at fixed T	T, graph G
Simulated Annealing	single solution	decrease $T_k \searrow 0$; non-homogeneous MC	cooling schedule $\{T_k\}$
LIP	single solution	proposal weighted by π-ratio; balancing fn g	β, $g(\cdot)$, M
Cross-Entropy	distribution $\boldsymbol{\theta}$	MLE update over elite samples; $\gamma_t \nearrow$	M, ρ, α
Metropolis-CE	solution + distribution	CE update of the transition distribution	all of the above

Chapter 7 Summary II

Theoretical guarantees.

- **Metropolis at fixed T:** the chain has stationary distribution π^T (the Boltzmann distribution). The mode of π^T is the minimiser of H. No convergence guarantee for finite runs; the chain may remain trapped in local minima.
- **Simulated Annealing:** converges to the global optimum D^* with probability 1 if and only if the cooling satisfies $\sum_k e^{-c/T_k} = \infty$ (Hajék's condition). Logarithmic cooling $T_k = d/\log(k)$ satisfies this; geometric cooling does not.
- **LIP:** has the same stationary distribution as Metropolis (the Boltzmann π^T), but explores more efficiently by using local gradient information. No additional convergence guarantees beyond standard Metropolis.
- **CE and Metropolis-CE:** heuristic algorithms with no general convergence guarantees, but empirically the most effective on both benchmarks studied here.

Practical recommendations for a new problem.

1. Start with plain Metropolis to verify implementation correctness and understand the landscape.

Chapter 7 Summary III

2. If Metropolis gets trapped: try SA with geometric cooling $T_k = 0.99^k T_0$.
3. For better quality with moderate budget: try LIP (better solutions, $\sim M\times$ more expensive per step).
4. For best solution quality with larger budget: use CE or Metropolis-CE. CE provides the richest distributional model but is slowest; Metropolis-CE is $\approx 20\times$ faster with comparable quality.

Unifying Theme: Probability as a Search Tool

All five methods use the Boltzmann distribution as a bridge between probability and optimisation. The core principle: assign higher probability to better solutions, then *sample* efficiently from that distribution. The methods differ only in how they implement this sampling, trading off computational cost per step against the quality of exploration.